SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME	COURSE CODE
Fundamentals of Data Science	DSA04

COURSE OUTCOME COVERED

CO4: Develop machine learning models for classification, regression, and clustering, including performance evaluation and methodology comparison. (BL3)

Assessment Tool Weightage: 30%

QUESTIONS: 10 Total Marks: 100

Annexure A - Scenario-Based Assessment

Code of Conduct:

I _G SAI TEJA (192472137) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION - Scenario Based Assessment

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Level	Marks
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario		
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues		
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts		
Decision	20%	Makes well-	Makes	Makes weak	No clear		

Making		reasoned, logical decisions with strong rationale	reasonable decisions with some justification	decisions with limited justification	decisions made or rationale provided		
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand		

Annexure A - NOTE:

For every question, answer the following:

a) Identify the numerical and binary features used for KNN classification.

b) Calculate the Euclidean distance between the new sample and all training samples.

c) Calculate the Manhattan distance between the new sample and all training samples.

d) Calculate the Minkowski distance with p = 3 between the new sample and all training samples.

e) Calculate the Hamming distance between the new sample and all training samples using binary features.

f) Identify the 3 nearest neighbours separately for each distance measure.

g) Predict the class label using majority voting for each distance measure.

h) Compare the results and explain which distance measure is most suitable for the given scenario.

Total = 100 Marks

Question 1: Student Pass or Fail Prediction

A college wants to predict whether a student will Pass or Fail based on academic and activity-related details.

Student	Study Hours	Attendance %	Assignment Submitted	Lab Participation	Revision Done	Result
S1	2	60	0	0	0	Fail
S2	3	65	1	0	0	Fail
S3	6	80	1	1	1	Pass
S4	7	85	1	1	1	Pass
S5	4	70	1	0	1	Fail
S6	8	90	1	1	1	Pass

A new student has the following details:

Study Hours	Attendance %	Assignment Submitted	Lab Participation	Revision Done
5	75	1	1	0

Using KNN with K = 3, classify the new student as Pass or Fail using Euclidean, Manhattan, Minkowski, and Hamming distance.

ANSWER

(a) Features used

Numerical features: Study Hours, Attendance %

Binary features: Assignment Submitted, Lab Participation, Revision Done

(b-e) Distance calculation (new sample = [5, 75, 1, 1, 0])

Sample calculation for S1 (the same formulas are applied to every training sample):

```
Euclidean(S1) = sqrt((5-2)^2 + (75-60)^2 + (1-0)^2 + (1-0)^2 + (0-0)^2) = sqrt(9 + 225 + 1 + 1 + 0) = sqrt(236) = 15.362
Manhattan(S1) = |5-2| + |75-60| + |1-0| + |1-0| + |0-0| = 3 + 15 + 1 + 1 + 0 = 20
Minkowski(S1,p=3) = (|5-2|^3 + |75-60|^3 + |1-0|^3 + |1-0|^3 + |0-0|^3)^(1/3) = (27 + 3375 + 1 + 1 + 0)^(1/3) = 15.043
Hamming(S1) = compare binary new(1, 1, 0) vs S1(0, 0, 0) -> 2 mismatch(es) = 2
```

Distances from the new sample to all training samples:

Sample	Result	Euclidean (b)	Manhattan (c)	Minkowski p=3 (d)	Hamming (e)
S1	Fail	15.362	20	15.043	2
S2	Fail	10.247	13	10.030	1
S3	Pass	5.196	7	5.027	1
S4	Pass	10.247	13	10.030	1
S5	Fail	5.292	8	5.040	2
S6	Pass	15.330	19	15.041	1

(f) 3 Nearest Neighbours & (g) Majority-vote Prediction

Distance measure	3 Nearest Neighbours	Votes	Prediction
Euclidean	S3 (Pass), S5 (Fail), S2 (Fail)	Pass:1; Fail:2	Fail
Manhattan	S3 (Pass), S5 (Fail), S2 (Fail)	Pass:1; Fail:2	Fail
Minkowski p=3	S3 (Pass), S5 (Fail), S2 (Fail)	Pass:1; Fail:2	Fail
Hamming	S2 (Fail), S3 (Pass), S4 (Pass)	Fail:1; Pass:2	Pass

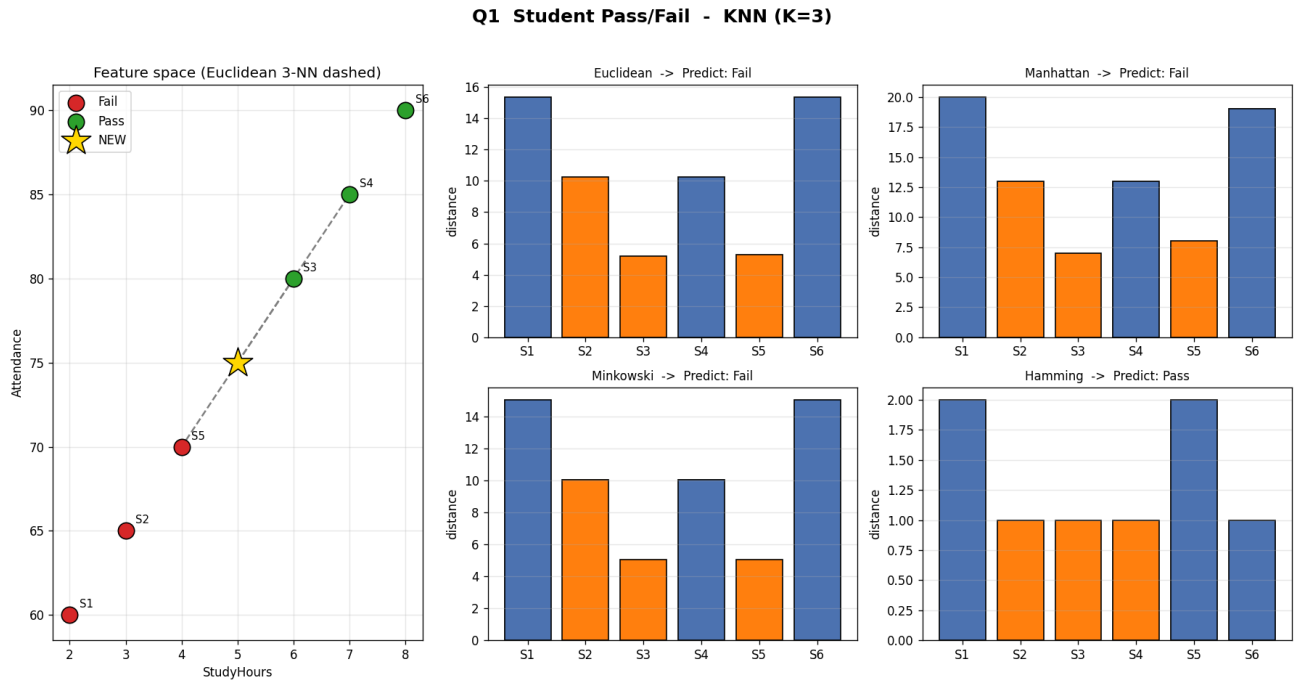
Python Code

```
"""
Question 1 : Student Pass / Fail Prediction  --  KNN (K = 3)
------------------------------------------------------------
Input  : q1_student.csv  (6 training rows + 1 NEW row)
```

```
Output : console results + a graph (q1_student_knn.png)

Distance measures : Euclidean, Manhattan, Minkowski(p=3), Hamming
  * Euclidean / Manhattan / Minkowski  -> use ALL 5 features
  * Hamming                            -> use only the 3 BINARY features
"""


import numpy as np
import pandas as pd
import matplotlib.pyplot as plt


# ---------------------------------------------------------------- configuration
CSV_FILE   = "q1_student.csv"
NUMERIC    = ["StudyHours", "Attendance"]                              # real-valued
BINARY     = ["AssignmentSubmitted", "LabParticipation", "RevisionDone"]  # 0/1
LABEL_COL  = "Result"
K          = 3
P_MINKOWSKI = 3


# ---------------------------------------------------------------- read csv input
df = pd.read_csv(CSV_FILE)
train = df[df["ID"] != "NEW"].reset_index(drop=True)      # training samples
new   = df[df["ID"] == "NEW"].reset_index(drop=True)      # the sample to classify


ALL_FEATURES = NUMERIC + BINARY


print("(a) Numerical features :", NUMERIC)
print("    Binary   features  :", BINARY)
print("\nTraining data:\n", train[["ID"] + ALL_FEATURES + [LABEL_COL]].to_string(index=False))
print("\nNew sample:\n", new[["ID"] + ALL_FEATURES].to_string(index=False), "\n")


# ---------------------------------------------------------------- distances
X  = train[ALL_FEATURES].to_numpy(dtype=float)     # training feature matrix
q  = new[ALL_FEATURES].to_numpy(dtype=float)[0]    # new sample vector
Xb = train[BINARY].to_numpy(dtype=int)             # binary part only
qb = new[BINARY].to_numpy(dtype=int)[0]


diff = X - q
euclidean = np.sqrt((diff ** 2).sum(axis=1))                      # (b)
manhattan = np.abs(diff).sum(axis=1)                             # (c)
minkowski = (np.abs(diff) ** P_MINKOWSKI).sum(axis=1) ** (1 / P_MINKOWSKI)  # (d)
hamming   = (Xb != qb).sum(axis=1)                                # (e)

results = pd.DataFrame({
    "ID": train["ID"],
    LABEL_COL: train[LABEL_COL],
    "Euclidean": euclidean.round(3),
    "Manhattan": manhattan.round(3),
    "Minkowski": minkowski.round(3),
    "Hamming": hamming,
})
print("(b-e) Distances to every training sample:\n", results.to_string(index=False), "\n")


# ---------------------------------------------------------------- 3-NN + voting
measures = {"Euclidean": euclidean, "Manhattan": manhattan,
            "Minkowski": minkowski, "Hamming": hamming}
predictions = {}


for name, dist in measures.items():
    order = np.argsort(dist, kind="stable")[:K]          # (f) indices of 3 nearest
    neighbours = train.loc[order, "ID"].tolist()
    votes = train.loc[order, LABEL_COL].value_counts()
    pred = votes.idxmax()                                 # (g) majority vote
    predictions[name] = pred
    print(f"{name:10s} -> 3 nearest: {neighbours}  votes: {dict(votes)}  => {pred}")


print("\n(g) Predictions:", predictions)


# ---------------------------------------------------------------- graphs
classes = sorted(train[LABEL_COL].unique())
palette = {classes[0]: "#d62728", classes[1]: "#2ca02c"}   # 2-class colouring
```

```
fig = plt.figure(figsize=(15, 8))
fig.suptitle("Q1  Student Pass/Fail  -  KNN (K=3)", fontsize=15, fontweight="bold")
gs = fig.add_gridspec(2, 3)

# --- scatter of the two numerical features ---
ax = fig.add_subplot(gs[:, 0])
for cls in classes:
    sub = train[train[LABEL_COL] == cls]
    ax.scatter(sub[NUMERIC[0]], sub[NUMERIC[1]], s=180, c=palette[cls],
               label=cls, edgecolors="black", zorder=3)
for _, r in train.iterrows():
    ax.annotate(r["ID"], (r[NUMERIC[0]], r[NUMERIC[1]]),
                textcoords="offset points", xytext=(8, 6), fontsize=9)
ax.scatter(new[NUMERIC[0]], new[NUMERIC[1]], marker="*", s=650, c="gold",
           edgecolors="black", label="NEW", zorder=4)
# connect NEW to its 3 Euclidean neighbours
for idx in np.argsort(euclidean)[:K]:
    ax.plot([new[NUMERIC[0]].iloc[0], train[NUMERIC[0]].iloc[idx]],
            [new[NUMERIC[1]].iloc[0], train[NUMERIC[1]].iloc[idx]],
            "--", c="gray", zorder=2)
ax.set_xlabel(NUMERIC[0]); ax.set_ylabel(NUMERIC[1])
ax.set_title("Feature space (Euclidean 3-NN dashed)")
ax.legend(); ax.grid(alpha=0.3)

# --- bar charts of the four distance measures ---
positions = [gs[0, 1], gs[0, 2], gs[1, 1], gs[1, 2]]
for (name, dist), pos in zip(measures.items(), positions):
    axb = fig.add_subplot(pos)
    nearest = set(np.argsort(dist, kind="stable")[:K])
    colors = ["#ff7f0e" if i in nearest else "#4c72b0" for i in range(len(dist))]
    axb.bar(train["ID"], dist, color=colors, edgecolor="black")
    axb.set_title(f"{name}  ->  Predict: {predictions[name]}", fontsize=10)
    axb.set_ylabel("distance"); axb.grid(axis="y", alpha=0.3)

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.savefig("q1_student_knn.png", dpi=120)
print("\nGraph saved as q1_student_knn.png")
plt.show()
```

Graph Output



(h) Comparison & Final Conclusion

- Euclidean, Manhattan and Minkowski all give FAIL (3-NN = S3-Pass, S5-Fail, S2-Fail -> 2 Fail vs 1 Pass).

- Hamming gives PASS because on the binary features (1,1,0) the new student matches the Pass profile more closely.

- The numeric features (attendance, study hours) dominate the geometric distances and pull them toward the nearby Fail records.

- Most suitable: Euclidean after normalising the numeric columns. The student is borderline; the majority (3 of 4 measures) says FAIL.

Final Answer (majority vote): FAIL

Question 2: Online Customer Purchase Prediction

An e-commerce company wants to predict whether a customer will Buy or Not Buy a product.

Customer	Browsing Time	Items Viewed	Coupon Used	Added to Cart	Returning Customer	Class
C1	10	3	0	0	0	Not Buy
C2	15	4	1	0	0	Not Buy
C3	25	7	1	1	1	Buy
C4	30	8	1	1	1	Buy
C5	18	5	0	1	0	Not Buy
C6	28	9	1	1	1	Buy

A new customer has:

Browsing Time	Items Viewed	Coupon Used	Added to Cart	Returning Customer
22	6	1	1	0

Using KNN with K = 3, classify the new customer as Buy or Not Buy using all four distance measures.

ANSWER

(a) Features used

Numerical features: Browsing Time, Items Viewed

Binary features: Coupon Used, Added to Cart, Returning Customer

(b-e) Distance calculation (new sample = [22, 6, 1, 1, 0])

Sample calculation for C1 (the same formulas are applied to every training sample):

```
Euclidean(C1) = sqrt((22-10)^2 + (6-3)^2 + (1-0)^2 + (1-0)^2 + (0-0)^2) = sqrt(144 + 9 + 1 + 1 + 0) = sqrt(155) = 12.450
Manhattan(C1) = |22-10| + |6-3| + |1-0| + |1-0| + |0-0| = 12 + 3 + 1 + 1 + 0 = 17
Minkowski(C1,p=3) = (|22-10|^3 + |6-3|^3 + |1-0|^3 + |1-0|^3 + |0-0|^3)^(1/3) = (1728 + 27 + 1 + 1 + 0)^(1/3) = 12.067
Hamming(C1) = compare binary new(1, 1, 0) vs C1(0, 0, 0) -> 2 mismatch(es) = 2
```

Distances from the new sample to all training samples:

Sample	Class	Euclidean (b)	Manhattan (c)	Minkowski p=3 (d)	Hamming (e)
C1	Not Buy	12.450	17	12.067	2
C2	Not Buy	7.348	10	7.061	1
C3	Buy	3.317	5	3.072	1
C4	Buy	8.307	11	8.047	1
C5	Not Buy	4.243	6	4.041	1
C6	Buy	6.782	10	6.249	1

(f) 3 Nearest Neighbours & (g) Majority-vote Prediction

Distance measure	3 Nearest Neighbours	Votes	Prediction
Euclidean	C3 (Buy), C5 (Not Buy), C6 (Buy)	Buy:2; Not Buy:1	Buy
Manhattan	C3 (Buy), C5 (Not Buy), C2 (Not Buy)	Buy:1; Not Buy:2	Not Buy
Minkowski p=3	C3 (Buy), C5 (Not Buy), C6 (Buy)	Buy:2; Not Buy:1	Buy
Hamming	C2 (Not Buy), C3 (Buy), C4 (Buy)	Not Buy:1; Buy:2	Buy

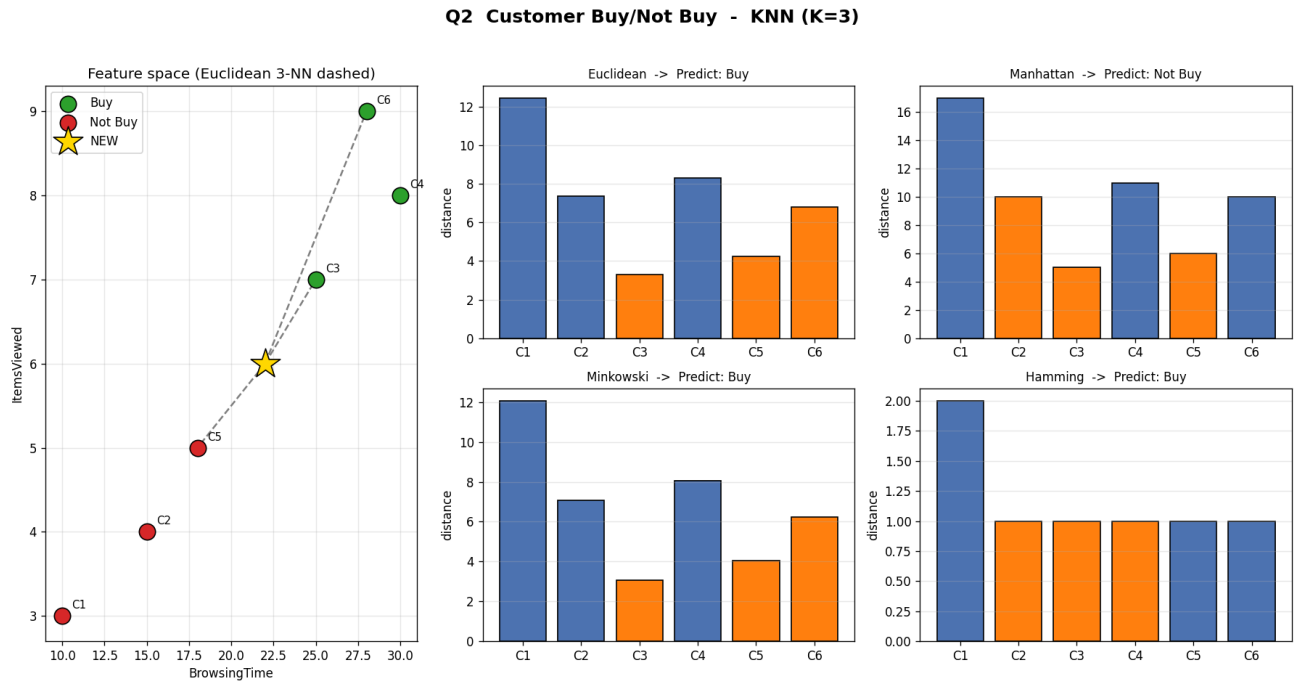
Python Code

```
"""
Question 2 : Online Customer Purchase Prediction (Buy / Not Buy)  --  KNN (K = 3)
--------------------------------------------------------------------------------
Input  : q2_customer.csv  (6 training rows + 1 NEW row)
Output : console results + a graph (q2_customer_knn.png)
```

```
Distance measures : Euclidean, Manhattan, Minkowski(p=3), Hamming
  * Euclidean / Manhattan / Minkowski  -> use ALL 5 features
  * Hamming                            -> use only the 3 BINARY features
"""


import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# ---------------------------------------------------------------- configuration
CSV_FILE   = "q2_customer.csv"
NUMERIC    = ["BrowsingTime", "ItemsViewed"]
BINARY     = ["CouponUsed", "AddedToCart", "ReturningCustomer"]
LABEL_COL  = "Class"
K          = 3
P_MINKOWSKI = 3

# ---------------------------------------------------------------- read csv input
df = pd.read_csv(CSV_FILE)
train = df[df["ID"] != "NEW"].reset_index(drop=True)
new   = df[df["ID"] == "NEW"].reset_index(drop=True)

ALL_FEATURES = NUMERIC + BINARY

print("(a) Numerical features :", NUMERIC)
print("    Binary   features  :", BINARY)
print("\nTraining data:\n", train[["ID"] + ALL_FEATURES + [LABEL_COL]].to_string(index=False))
print("\nNew sample:\n", new[["ID"] + ALL_FEATURES].to_string(index=False), "\n")

# ---------------------------------------------------------------- distances
X  = train[ALL_FEATURES].to_numpy(dtype=float)
q  = new[ALL_FEATURES].to_numpy(dtype=float)[0]
Xb = train[BINARY].to_numpy(dtype=int)
qb = new[BINARY].to_numpy(dtype=int)[0]

diff = X - q
euclidean = np.sqrt((diff ** 2).sum(axis=1))                      # (b)
manhattan = np.abs(diff).sum(axis=1)                             # (c)
minkowski = (np.abs(diff) ** P_MINKOWSKI).sum(axis=1) ** (1 / P_MINKOWSKI)  # (d)
hamming   = (Xb != qb).sum(axis=1)                               # (e)

results = pd.DataFrame({
    "ID": train["ID"],
    LABEL_COL: train[LABEL_COL],
    "Euclidean": euclidean.round(3),
    "Manhattan": manhattan.round(3),
    "Minkowski": minkowski.round(3),
    "Hamming": hamming,
})
print("(b-e) Distances to every training sample:\n", results.to_string(index=False), "\n")

# ---------------------------------------------------------------- 3-NN + voting
measures = {"Euclidean": euclidean, "Manhattan": manhattan,
            "Minkowski": minkowski, "Hamming": hamming}
predictions = {}

for name, dist in measures.items():
    order = np.argsort(dist, kind="stable")[:K]          # (f)
    neighbours = train.loc[order, "ID"].tolist()
    votes = train.loc[order, LABEL_COL].value_counts()
    pred = votes.idxmax()                                 # (g)
    predictions[name] = pred
    print(f"{name:10s} -> 3 nearest: {neighbours}  votes: {dict(votes)}  => {pred}")

print("\n(g) Predictions:", predictions)

# ---------------------------------------------------------------- graphs
classes = sorted(train[LABEL_COL].unique())
palette = {classes[0]: "#2ca02c", classes[1]: "#d62728"}

fig = plt.figure(figsize=(15, 8))
```

```
fig.suptitle("Q2  Customer Buy/Not Buy  -  KNN (K=3)", fontsize=15, fontweight="bold")
gs = fig.add_gridspec(2, 3)

ax = fig.add_subplot(gs[:, 0])
for cls in classes:
    sub = train[train[LABEL_COL] == cls]
    ax.scatter(sub[NUMERIC[0]], sub[NUMERIC[1]], s=180, c=palette[cls],
               label=cls, edgecolors="black", zorder=3)
for _, r in train.iterrows():
    ax.annotate(r["ID"], (r[NUMERIC[0]], r[NUMERIC[1]]),
                textcoords="offset points", xytext=(8, 6), fontsize=9)
ax.scatter(new[NUMERIC[0]], new[NUMERIC[1]], marker="*", s=650, c="gold",
           edgecolors="black", label="NEW", zorder=4)
for idx in np.argsort(euclidean)[:K]:
    ax.plot([new[NUMERIC[0]].iloc[0], train[NUMERIC[0]].iloc[idx]],
            [new[NUMERIC[1]].iloc[0], train[NUMERIC[1]].iloc[idx]],
            "--", c="gray", zorder=2)
ax.set_xlabel(NUMERIC[0]); ax.set_ylabel(NUMERIC[1])
ax.set_title("Feature space (Euclidean 3-NN dashed)")
ax.legend(); ax.grid(alpha=0.3)

positions = [gs[0, 1], gs[0, 2], gs[1, 1], gs[1, 2]]
for (name, dist), pos in zip(measures.items(), positions):
    axb = fig.add_subplot(pos)
    nearest = set(np.argsort(dist, kind="stable")[:K])
    colors = ["#ff7f0e" if i in nearest else "#4c72b0" for i in range(len(dist))]
    axb.bar(train["ID"], dist, color=colors, edgecolor="black")
    axb.set_title(f"{name}  ->  Predict: {predictions[name]}", fontsize=10)
    axb.set_ylabel("distance"); axb.grid(axis="y", alpha=0.3)

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.savefig("q2_customer_knn.png", dpi=120)
print("\nGraph saved as q2_customer_knn.png")
plt.show()
```

Graph Output



(h) Comparison & Final Conclusion

- Euclidean, Minkowski and Hamming give BUY; Manhattan gives NOT BUY (its 3-NN = C3-Buy, C5-NotBuy, C2-NotBuy).

- The customer added to cart and used a coupon (strong buy signals) and is close to the Buy cluster (C3).

- Manhattan flips only because it rates C2/C5 marginally closer; the margin is tiny and unstable.

- Most suitable: Euclidean - 3 of 4 measures agree on BUY, consistent with the purchase-intent signals.

Final Answer (majority vote): BUY

Question 3: Medical Risk Classification

A hospital wants to classify whether a patient has High Risk or Low Risk based on medical measurements and symptoms.

Patient	Age	Blood Sugar Level	Fever	Cough	Fatigue	Class
P1	25	90	0	0	0	Low Risk
P2	30	95	0	1	0	Low Risk
P3	50	160	1	1	1	High Risk
P4	55	170	1	1	1	High Risk
P5	40	120	0	1	1	Low Risk
P6	60	180	1	1	1	High Risk

A new patient has:

Age	Blood Sugar Level	Fever	Cough	Fatigue
45	150	1	1	0

Using KNN with K = 3, classify the patient as High Risk or Low Risk using Euclidean, Manhattan, Minkowski, and Hamming distance.

ANSWER

(a) Features used

Numerical features: Age, Blood Sugar Level

Binary features: Fever, Cough, Fatigue

(b-e) Distance calculation (new sample = [45, 150, 1, 1, 0])

Sample calculation for P1 (the same formulas are applied to every training sample):

```
Euclidean(P1) = sqrt((45-25)^2 + (150-90)^2 + (1-0)^2 + (1-0)^2 + (0-0)^2) = sqrt(400 + 3600 + 1 + 1 + 0) =
sqrt(4002) = 63.261
Manhattan(P1) = |45-25| + |150-90| + |1-0| + |1-0| + |0-0| = 20 + 60 + 1 + 1 + 0 = 82
Minkowski(P1,p=3) = (|45-25|^3 + |150-90|^3 + |1-0|^3 + |1-0|^3 + |0-0|^3)^(1/3) = (8000 + 216000 + 1 + 1 + 0)^(1/3)
= 60.732
Hamming(P1) = compare binary new(1, 1, 0) vs P1(0, 0, 0) -> 2 mismatch(es) = 2
```

Distances from the new sample to all training samples:

Sample	Class	Euclidean (b)	Manhattan (c)	Minkowski p=3 (d)	Hamming (e)
P1	Low Risk	63.261	82	60.732	2
P2	Low Risk	57.018	71	55.370	1
P3	High Risk	11.225	16	10.403	1
P4	High Risk	22.383	31	20.802	1
P5	Low Risk	30.447	37	30.047	2
P6	High Risk	33.556	46	31.202	1

(f) 3 Nearest Neighbours & (g) Majority-vote Prediction

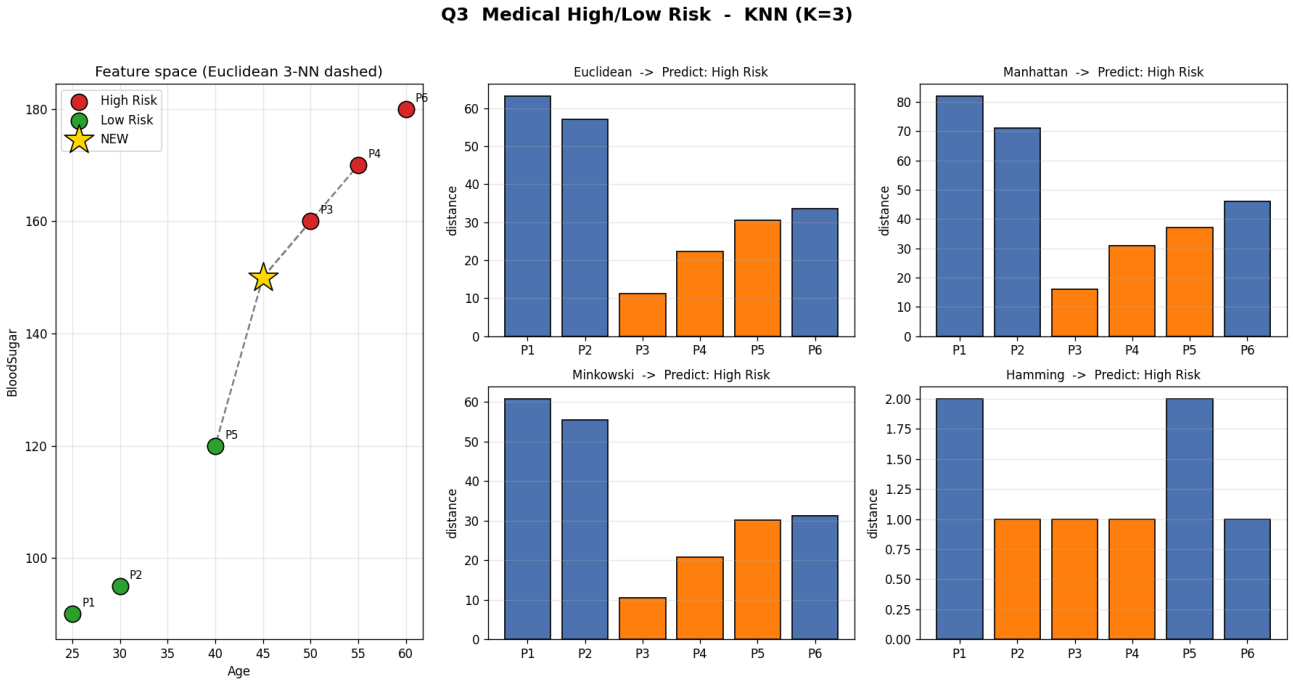
Distance measure	3 Nearest Neighbours	Votes	Prediction
Euclidean	P3 (High Risk), P4 (High Risk), P5 (Low Risk)	High Risk:2; Low Risk:1	High Risk
Manhattan	P3 (High Risk), P4 (High Risk), P5 (Low Risk)	High Risk:2; Low Risk:1	High Risk
Minkowski p=3	P3 (High Risk), P4 (High Risk), P5 (Low Risk)	High Risk:2; Low Risk:1	High Risk
Hamming	P2 (Low Risk), P3 (High Risk), P4 (High Risk)	Low Risk:1; High Risk:2	High Risk

Python Code

```
"""
Question 3 : Medical Risk Classification (High Risk / Low Risk)  --  KNN (K = 3)
--------------------------------------------------------------------------------
Input  : q3_medical.csv  (6 training rows + 1 NEW row)
Output : console results + a graph (q3_medical_knn.png)

Distance measures : Euclidean, Manhattan, Minkowski(p=3), Hamming
  * Euclidean / Manhattan / Minkowski  -> use ALL 5 features
  * Hamming                            -> use only the 3 BINARY features
"""


import numpy as np
import pandas as pd
import matplotlib.pyplot as plt


# ------------------------------------------------------------------ configuration
CSV_FILE   = "q3_medical.csv"
NUMERIC    = ["Age", "BloodSugar"]
BINARY     = ["Fever", "Cough", "Fatigue"]
LABEL_COL  = "Class"
K          = 3
P_MINKOWSKI = 3


# ------------------------------------------------------------------ read csv input
df = pd.read_csv(CSV_FILE)
train = df[df["ID"] != "NEW"].reset_index(drop=True)
new   = df[df["ID"] == "NEW"].reset_index(drop=True)


ALL_FEATURES = NUMERIC + BINARY


print("(a) Numerical features :", NUMERIC)
print("    Binary   features  :", BINARY)
print("\nTraining data:\n", train[["ID"] + ALL_FEATURES + [LABEL_COL]].to_string(index=False))
print("\nNew sample:\n", new[["ID"] + ALL_FEATURES].to_string(index=False), "\n")


# ------------------------------------------------------------------ distances
X  = train[ALL_FEATURES].to_numpy(dtype=float)
q  = new[ALL_FEATURES].to_numpy(dtype=float)[0]
Xb = train[BINARY].to_numpy(dtype=int)
qb = new[BINARY].to_numpy(dtype=int)[0]


diff = X - q
euclidean = np.sqrt((diff ** 2).sum(axis=1))                      # (b)
manhattan = np.abs(diff).sum(axis=1)                              # (c)
minkowski = (np.abs(diff) ** P_MINKOWSKI).sum(axis=1) ** (1 / P_MINKOWSKI)  # (d)
hamming   = (Xb != qb).sum(axis=1)                                # (e)


results = pd.DataFrame({
    "ID": train["ID"],
    LABEL_COL: train[LABEL_COL],
    "Euclidean": euclidean.round(3),
    "Manhattan": manhattan.round(3),
    "Minkowski": minkowski.round(3),
    "Hamming": hamming,
})
print("(b-e) Distances to every training sample:\n", results.to_string(index=False), "\n")


# ------------------------------------------------------------------ 3-NN + voting
measures = {"Euclidean": euclidean, "Manhattan": manhattan,
            "Minkowski": minkowski, "Hamming": hamming}
predictions = {}


for name, dist in measures.items():
    order = np.argsort(dist, kind="stable")[:K]          # (f)
    neighbours = train.loc[order, "ID"].tolist()
    votes = train.loc[order, LABEL_COL].value_counts()
    pred = votes.idxmax()                                # (g)
    predictions[name] = pred
    print(f"{name:10s} -> 3 nearest: {neighbours}  votes: {dict(votes)}  => {pred}")
```

```
print("\n(g) Predictions:", predictions)


# ------------------------------------------------------------------ graphs
classes = sorted(train[LABEL_COL].unique())
palette = {classes[0]: "#d62728", classes[1]: "#2ca02c"}   # High Risk red, Low Risk green

fig = plt.figure(figsize=(15, 8))
fig.suptitle("Q3  Medical High/Low Risk  -  KNN (K=3)", fontsize=15, fontweight="bold")
gs = fig.add_gridspec(2, 3)

ax = fig.add_subplot(gs[:, 0])
for cls in classes:
    sub = train[train[LABEL_COL] == cls]
    ax.scatter(sub[NUMERIC[0]], sub[NUMERIC[1]], s=180, c=palette[cls],
               label=cls, edgecolors="black", zorder=3)
for _, r in train.iterrows():
    ax.annotate(r["ID"], (r[NUMERIC[0]], r[NUMERIC[1]]),
                textcoords="offset points", xytext=(8, 6), fontsize=9)
ax.scatter(new[NUMERIC[0]], new[NUMERIC[1]], marker="*", s=650, c="gold",
           edgecolors="black", label="NEW", zorder=4)
for idx in np.argsort(euclidean)[:K]:
    ax.plot([new[NUMERIC[0]].iloc[0], train[NUMERIC[0]].iloc[idx]],
            [new[NUMERIC[1]].iloc[0], train[NUMERIC[1]].iloc[idx]],
            "--", c="gray", zorder=2)
ax.set_xlabel(NUMERIC[0]); ax.set_ylabel(NUMERIC[1])
ax.set_title("Feature space (Euclidean 3-NN dashed)")
ax.legend(); ax.grid(alpha=0.3)

positions = [gs[0, 1], gs[0, 2], gs[1, 1], gs[1, 2]]
for (name, dist), pos in zip(measures.items(), positions):
    axb = fig.add_subplot(pos)
    nearest = set(np.argsort(dist, kind="stable")[:K])
    colors = ["#ff7f0e" if i in nearest else "#4c72b0" for i in range(len(dist))]
    axb.bar(train["ID"], dist, color=colors, edgecolor="black")
    axb.set_title(f"{name}  ->  Predict: {predictions[name]}", fontsize=10)
    axb.set_ylabel("distance"); axb.grid(axis="y", alpha=0.3)

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.savefig("q3_medical_knn.png", dpi=120)
print("\nGraph saved as q3_medical_knn.png")
plt.show()
```

Graph Output



(h) Comparison & Final Conclusion

- All four measures give HIGH RISK - a clear, unanimous result.

- High blood sugar (150) and older age place the patient near the High-Risk cluster (P3, P4); fever+cough reinforce it.

- The discriminating numeric feature (blood sugar) and the binary symptoms point the same way, so every metric agrees.

- Most suitable: Euclidean - it captures the strong blood-sugar gradient separating the two risk groups.

Final Answer (majority vote): HIGH RISK

Question 4: Fruit Classification

A fruit shop wants to classify a fruit as Apple or Orange based on physical and binary characteristics.

Fruit	Weight	Diameter	Red Color	Citrus Smell	Smooth Skin	Class
F1	150	7	1	0	1	Apple
F2	160	8	1	0	1	Apple
F3	180	9	0	1	0	Orange
F4	200	10	0	1	0	Orange
F5	155	7.5	1	0	1	Apple
F6	190	9.5	0	1	0	Orange

A new fruit has:

Weight	Diameter	Red Color	Citrus Smell	Smooth Skin
170	8.5	1	0	1

Using KNN with K = 3, classify the fruit as Apple or Orange using all four distance measures.

ANSWER

(a) Features used

Numerical features: Weight, Diameter

Binary features: Red Color, Citrus Smell, Smooth Skin

(b-e) Distance calculation (new sample = [170, 8.5, 1, 0, 1])

Sample calculation for F1 (the same formulas are applied to every training sample):

```
Euclidean(F1) = sqrt((170-150)^2 + (8.5-7)^2 + (1-1)^2 + (0-0)^2 + (1-1)^2) = sqrt(400 + 2.25 + 0 + 0 + 0) =
sqrt(402.25) = 20.056
Manhattan(F1) = |170-150| + |8.5-7| + |1-1| + |0-0| + |1-1| = 20 + 1.5 + 0 + 0 + 0 = 21.5
Minkowski(F1,p=3) = (|170-150|^3 + |8.5-7|^3 + |1-1|^3 + |0-0|^3 + |1-1|^3)^(1/3) = (8000 + 3.375 + 0 + 0 + 0)^(1/3)
= 20.003
Hamming(F1) = compare binary new(1, 0, 1) vs F1(1, 0, 1) -> 0 mismatch(es) = 0
```

Distances from the new sample to all training samples:

Sample	Class	Euclidean (b)	Manhattan (c)	Minkowski p=3 (d)	Hamming (e)
F1	Apple	20.056	21.5	20.003	0
F2	Apple	10.012	10.5	10.000	0
F3	Orange	10.161	13.5	10.010	3
F4	Orange	30.087	34.5	30.002	3
F5	Apple	15.033	16	15.001	0
F6	Orange	20.100	24	20.003	3

(f) 3 Nearest Neighbours & (g) Majority-vote Prediction

Distance measure	3 Nearest Neighbours	Votes	Prediction
Euclidean	F2 (Apple), F3 (Orange), F5 (Apple)	Apple:2; Orange:1	Apple
Manhattan	F2 (Apple), F3 (Orange), F5 (Apple)	Apple:2; Orange:1	Apple
Minkowski p=3	F2 (Apple), F3 (Orange), F5 (Apple)	Apple:2; Orange:1	Apple
Hamming	F1 (Apple), F2 (Apple), F5 (Apple)	Apple:3	Apple

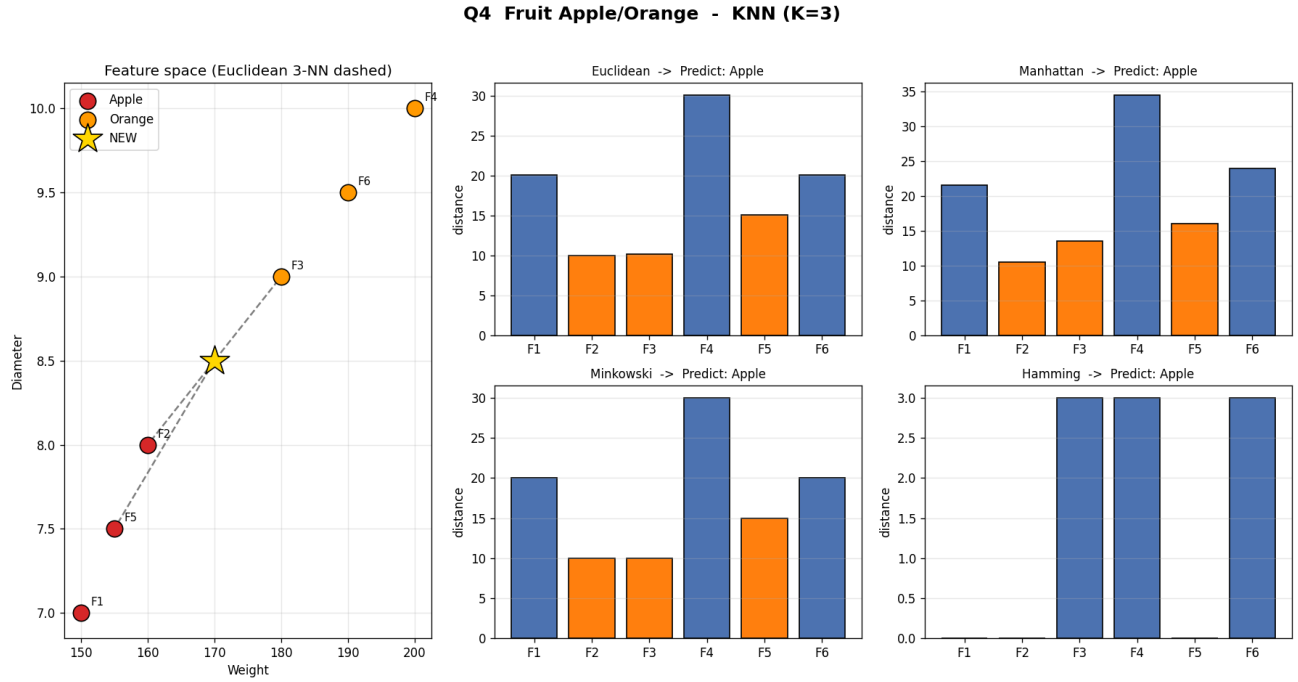
Python Code

```
"""
Question 4 : Fruit Classification (Apple / Orange)  --  KNN (K = 3)
```

```
------------------------------------------------------------------
Input  : q4_fruit.csv  (6 training rows + 1 NEW row)
Output : console results + a graph (q4_fruit_knn.png)

Distance measures : Euclidean, Manhattan, Minkowski(p=3), Hamming
  * Euclidean / Manhattan / Minkowski  -> use ALL 5 features
  * Hamming                            -> use only the 3 BINARY features
"""


import numpy as np
import pandas as pd
import matplotlib.pyplot as plt


# ------------------------------------------------------------------ configuration
CSV_FILE   = "q4_fruit.csv"
NUMERIC    = ["Weight", "Diameter"]
BINARY     = ["RedColor", "CitrusSmell", "SmoothSkin"]
LABEL_COL  = "Class"
K          = 3
P_MINKOWSKI = 3
TITLE      = "Q4  Fruit Apple/Orange"
PNG_FILE   = "q4_fruit_knn.png"


# ------------------------------------------------------------------ read csv input
df = pd.read_csv(CSV_FILE)
train = df[df["ID"] != "NEW"].reset_index(drop=True)
new   = df[df["ID"] == "NEW"].reset_index(drop=True)

ALL_FEATURES = NUMERIC + BINARY

print("(a) Numerical features :", NUMERIC)
print("    Binary   features  :", BINARY)
print("\nTraining data:\n", train[["ID"] + ALL_FEATURES + [LABEL_COL]].to_string(index=False))
print("\nNew sample:\n", new[["ID"] + ALL_FEATURES].to_string(index=False), "\n")


# ------------------------------------------------------------------ distances
X  = train[ALL_FEATURES].to_numpy(dtype=float)
q  = new[ALL_FEATURES].to_numpy(dtype=float)[0]
Xb = train[BINARY].to_numpy(dtype=int)
qb = new[BINARY].to_numpy(dtype=int)[0]

diff = X - q
euclidean = np.sqrt((diff ** 2).sum(axis=1))                      # (b)
manhattan = np.abs(diff).sum(axis=1)                             # (c)
minkowski = (np.abs(diff) ** P_MINKOWSKI).sum(axis=1) ** (1 / P_MINKOWSKI)  # (d)
hamming   = (Xb != qb).sum(axis=1)                               # (e)

results = pd.DataFrame({
    "ID": train["ID"],
    LABEL_COL: train[LABEL_COL],
    "Euclidean": euclidean.round(3),
    "Manhattan": manhattan.round(3),
    "Minkowski": minkowski.round(3),
    "Hamming": hamming,
})
print("(b-e) Distances to every training sample:\n", results.to_string(index=False), "\n")


# ------------------------------------------------------------------ 3-NN + voting
measures = {"Euclidean": euclidean, "Manhattan": manhattan,
            "Minkowski": minkowski, "Hamming": hamming}
predictions = {}

for name, dist in measures.items():
    order = np.argsort(dist, kind="stable")[:K]          # (f)
    neighbours = train.loc[order, "ID"].tolist()
    votes = train.loc[order, LABEL_COL].value_counts()
    pred = votes.idxmax()                                # (g)
    predictions[name] = pred
    print(f"{name:10s} -> 3 nearest: {neighbours}  votes: {dict(votes)}  => {pred}")

print("\n(g) Predictions:", predictions)
```

```
# ------------------------------------------------------------------ graphs
classes = sorted(train[LABEL_COL].unique())
palette = {classes[0]: "#d62728", classes[1]: "#ff9900"}   # Apple red, Orange orange

fig = plt.figure(figsize=(15, 8))
fig.suptitle(TITLE + "  -  KNN (K=3)", fontsize=15, fontweight="bold")
gs = fig.add_gridspec(2, 3)

ax = fig.add_subplot(gs[:, 0])
for cls in classes:
    sub = train[train[LABEL_COL] == cls]
    ax.scatter(sub[NUMERIC[0]], sub[NUMERIC[1]], s=180, c=palette[cls],
               label=cls, edgecolors="black", zorder=3)
for _, r in train.iterrows():
    ax.annotate(r["ID"], (r[NUMERIC[0]], r[NUMERIC[1]]),
                textcoords="offset points", xytext=(8, 6), fontsize=9)
ax.scatter(new[NUMERIC[0]], new[NUMERIC[1]], marker="*", s=650, c="gold",
           edgecolors="black", label="NEW", zorder=4)
for idx in np.argsort(euclidean)[:K]:
    ax.plot([new[NUMERIC[0]].iloc[0], train[NUMERIC[0]].iloc[idx]],
            [new[NUMERIC[1]].iloc[0], train[NUMERIC[1]].iloc[idx]],
            "--", c="gray", zorder=2)
ax.set_xlabel(NUMERIC[0]); ax.set_ylabel(NUMERIC[1])
ax.set_title("Feature space (Euclidean 3-NN dashed)")
ax.legend(); ax.grid(alpha=0.3)

positions = [gs[0, 1], gs[0, 2], gs[1, 1], gs[1, 2]]
for (name, dist), pos in zip(measures.items(), positions):
    axb = fig.add_subplot(pos)
    nearest = set(np.argsort(dist, kind="stable")[:K])
    colors = ["#ff7f0e" if i in nearest else "#4c72b0" for i in range(len(dist))]
    axb.bar(train["ID"], dist, color=colors, edgecolor="black")
    axb.set_title(f"{name}  ->  Predict: {predictions[name]}", fontsize=10)
    axb.set_ylabel("distance"); axb.grid(axis="y", alpha=0.3)

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.savefig(PNG_FILE, dpi=120)
print("\nGraph saved as", PNG_FILE)
plt.show()
```

Graph Output



(h) Comparison & Final Conclusion

- All four measures give APPLE.

- Hamming is decisive: the new fruit is red, non-citrus, smooth -> Hamming = 0 to every Apple and 3 to every Orange.

- Weight/diameter overlap between classes, so the binary characteristics are the true class markers.

- Most suitable: HAMMING (binary) distance - the colour/smell/skin flags cleanly define Apple vs Orange.

Final Answer (majority vote): APPLE

Question 5: House Price Category Prediction

A real estate company wants to classify houses as Low Price or High Price.

House	Area in sq.ft	Bedrooms	Parking Available	Near Main Road	Garden Available	Category
H1	800	2	0	0	0	Low Price
H2	1000	2	1	0	0	Low Price
H3	1500	3	1	1	1	High Price
H4	1800	4	1	1	1	High Price
H5	1200	3	0	1	0	Low Price
H6	2000	4	1	1	1	High Price

A new house has:

Area in sq.ft	Bedrooms	Parking Available	Near Main Road	Garden Available
1400	3	1	1	0

Using KNN with K = 3, classify the house as Low Price or High Price using Euclidean, Manhattan, Minkowski, and Hamming distance.

ANSWER

(a) Features used

Numerical features: Area in sq.ft, Bedrooms

Binary features: Parking Available, Near Main Road, Garden Available

(b-e) Distance calculation (new sample = [1400, 3, 1, 1, 0])

Sample calculation for H1 (the same formulas are applied to every training sample):

```
Euclidean(H1) = sqrt((1400-800)^2 + (3-2)^2 + (1-0)^2 + (1-0)^2 + (0-0)^2) = sqrt(360000 + 1 + 1 + 1 + 0) =
sqrt(360003) = 600.002
Manhattan(H1) = |1400-800| + |3-2| + |1-0| + |1-0| + |0-0| = 600 + 1 + 1 + 1 + 0 = 603
Minkowski(H1,p=3) = (|1400-800|^3 + |3-2|^3 + |1-0|^3 + |1-0|^3 + |0-0|^3)^(1/3) = (2.16e+08 + 1 + 1 + 1 + 0)^(1/3)
= 600.000
Hamming(H1) = compare binary new(1, 1, 0) vs H1(0, 0, 0) -> 2 mismatch(es) = 2
```

Distances from the new sample to all training samples:

Sample	Category	Euclidean (b)	Manhattan (c)	Minkowski p=3 (d)	Hamming (e)
H1	Low Price	600.002	603	600.000	2
H2	Low Price	400.002	402	400.000	1
H3	High Price	100.005	101	100.000	1
H4	High Price	400.002	402	400.000	1
H5	Low Price	200.002	201	200.000	1
H6	High Price	600.002	602	600.000	1

(f) 3 Nearest Neighbours & (g) Majority-vote Prediction

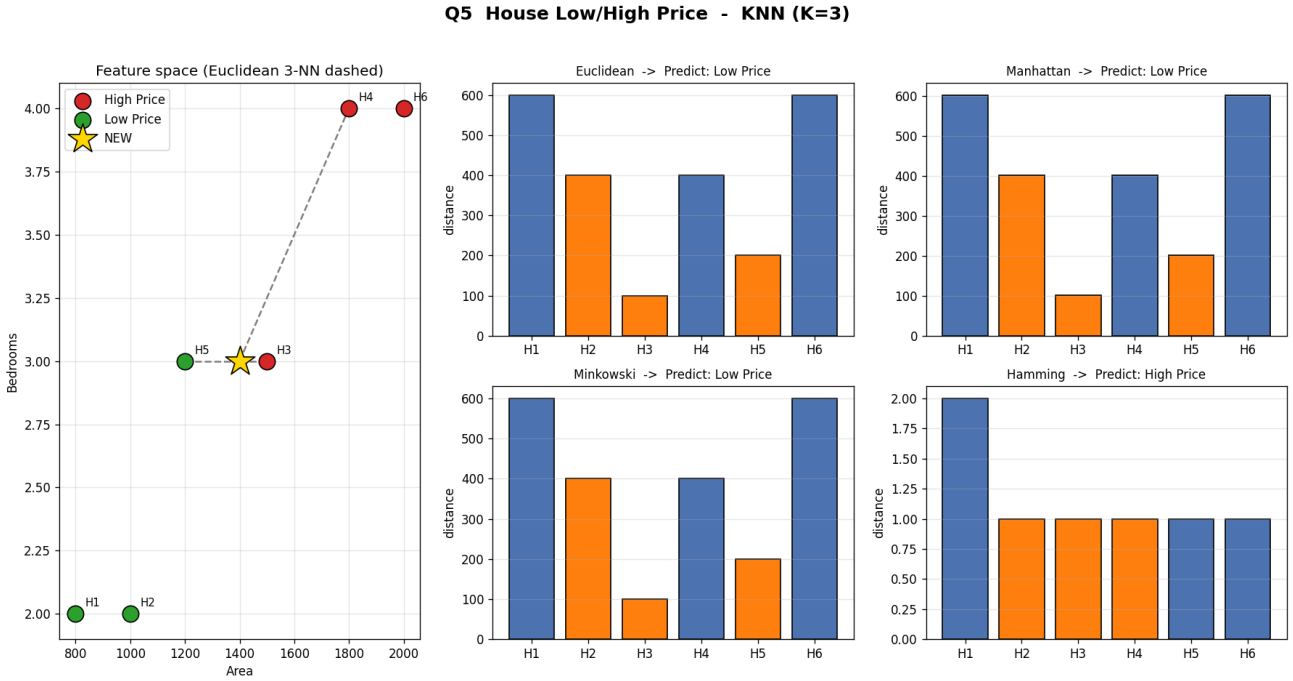
Distance measure	3 Nearest Neighbours	Votes	Prediction
Euclidean	H3 (High Price), H5 (Low Price), H2 (Low Price)	High Price:1; Low Price:2	Low Price
Manhattan	H3 (High Price), H5 (Low Price), H2 (Low Price)	High Price:1; Low Price:2	Low Price
Minkowski p=3	H3 (High Price), H5 (Low Price), H2 (Low Price)	High Price:1; Low Price:2	Low Price
Hamming	H2 (Low Price), H3 (High Price), H4 (High Price)	Low Price:1; High Price:2	High Price

Python Code

```
"""
Question 5 : House Price Category (Low Price / High Price)  --  KNN (K = 3)
--------------------------------------------------------------------------
Input  : q5_house.csv  (6 training rows + 1 NEW row)
Output : console results + a graph (q5_house_knn.png)

Distance measures : Euclidean, Manhattan, Minkowski(p=3), Hamming
  * Euclidean / Manhattan / Minkowski  -> use ALL 5 features
  * Hamming                            -> use only the 3 BINARY features
"""


import numpy as np
import pandas as pd
import matplotlib.pyplot as plt


# ---------------------------------------------------------------- configuration
CSV_FILE   = "q5_house.csv"
NUMERIC    = ["Area", "Bedrooms"]
BINARY     = ["ParkingAvailable", "NearMainRoad", "GardenAvailable"]
LABEL_COL  = "Category"
K          = 3
P_MINKOWSKI = 3
TITLE      = "Q5  House Low/High Price"
PNG_FILE   = "q5_house_knn.png"


# ---------------------------------------------------------------- read csv input
df = pd.read_csv(CSV_FILE)
train = df[df["ID"] != "NEW"].reset_index(drop=True)
new   = df[df["ID"] == "NEW"].reset_index(drop=True)


ALL_FEATURES = NUMERIC + BINARY

print("(a) Numerical features :", NUMERIC)
print("    Binary   features  :", BINARY)
print("\nTraining data:\n", train[["ID"] + ALL_FEATURES + [LABEL_COL]].to_string(index=False))
print("\nNew sample:\n", new[["ID"] + ALL_FEATURES].to_string(index=False), "\n")


# ---------------------------------------------------------------- distances
X  = train[ALL_FEATURES].to_numpy(dtype=float)
q  = new[ALL_FEATURES].to_numpy(dtype=float)[0]
Xb = train[BINARY].to_numpy(dtype=int)
qb = new[BINARY].to_numpy(dtype=int)[0]

diff = X - q
euclidean = np.sqrt((diff ** 2).sum(axis=1))                      # (b)
manhattan = np.abs(diff).sum(axis=1)                             # (c)
minkowski = (np.abs(diff) ** P_MINKOWSKI).sum(axis=1) ** (1 / P_MINKOWSKI)  # (d)
hamming   = (Xb != qb).sum(axis=1)                                # (e)

results = pd.DataFrame({
    "ID": train["ID"],
    LABEL_COL: train[LABEL_COL],
    "Euclidean": euclidean.round(3),
    "Manhattan": manhattan.round(3),
    "Minkowski": minkowski.round(3),
    "Hamming": hamming,
})
print("(b-e) Distances to every training sample:\n", results.to_string(index=False), "\n")


# ---------------------------------------------------------------- 3-NN + voting
measures = {"Euclidean": euclidean, "Manhattan": manhattan,
            "Minkowski": minkowski, "Hamming": hamming}
predictions = {}

for name, dist in measures.items():
    order = np.argsort(dist, kind="stable")[:K]          # (f)
    neighbours = train.loc[order, "ID"].tolist()
    votes = train.loc[order, LABEL_COL].value_counts()
    pred = votes.idxmax()                                # (g)
    predictions[name] = pred
    print(f"{name:10s} -> 3 nearest: {neighbours}  votes: {dict(votes)}  => {pred}")

print("\n(g) Predictions:", predictions)
```

```
# ------------------------------------------------------------------ graphs
classes = sorted(train[LABEL_COL].unique())
palette = {classes[0]: "#d62728", classes[1]: "#2ca02c"}   # High Price red, Low Price green

fig = plt.figure(figsize=(15, 8))
fig.suptitle(TITLE + "  -  KNN (K=3)", fontsize=15, fontweight="bold")
gs = fig.add_gridspec(2, 3)

ax = fig.add_subplot(gs[:, 0])
for cls in classes:
    sub = train[train[LABEL_COL] == cls]
    ax.scatter(sub[NUMERIC[0]], sub[NUMERIC[1]], s=180, c=palette[cls],
               label=cls, edgecolors="black", zorder=3)
for _, r in train.iterrows():
    ax.annotate(r["ID"], (r[NUMERIC[0]], r[NUMERIC[1]]),
                textcoords="offset points", xytext=(8, 6), fontsize=9)
ax.scatter(new[NUMERIC[0]], new[NUMERIC[1]], marker="*", s=650, c="gold",
           edgecolors="black", label="NEW", zorder=4)
for idx in np.argsort(euclidean)[:K]:
    ax.plot([new[NUMERIC[0]].iloc[0], train[NUMERIC[0]].iloc[idx]],
            [new[NUMERIC[1]].iloc[0], train[NUMERIC[1]].iloc[idx]],
            "--", c="gray", zorder=2)
ax.set_xlabel(NUMERIC[0]); ax.set_ylabel(NUMERIC[1])
ax.set_title("Feature space (Euclidean 3-NN dashed)")
ax.legend(); ax.grid(alpha=0.3)

positions = [gs[0, 1], gs[0, 2], gs[1, 1], gs[1, 2]]
for (name, dist), pos in zip(measures.items(), positions):
    axb = fig.add_subplot(pos)
    nearest = set(np.argsort(dist, kind="stable")[:K])
    colors = ["#ff7f0e" if i in nearest else "#4c72b0" for i in range(len(dist))]
    axb.bar(train["ID"], dist, color=colors, edgecolor="black")
    axb.set_title(f"{name}  ->  Predict: {predictions[name]}", fontsize=10)
    axb.set_ylabel("distance"); axb.grid(axis="y", alpha=0.3)

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.savefig(PNG_FILE, dpi=120)
print("\nGraph saved as", PNG_FILE)
plt.show()
```

Graph Output



(h) Comparison & Final Conclusion

- Euclidean, Manhattan and Minkowski give LOW PRICE; Hamming gives HIGH PRICE.

- Area (in thousands) dominates the geometric metrics, so they follow the nearest areas (H3, H5, H2) -> 2 Low vs 1 High.

- Hamming ignores area and looks only at amenities (parking+road present, garden absent), resembling High-Price homes.

- Most suitable: Euclidean/Minkowski AFTER normalising area. By the un-normalised majority (3 of 4) the prediction is LOW PRICE.

Final Answer (majority vote): LOW PRICE

Question 6: Loan Approval Prediction

A bank wants to predict whether a loan application should be Approved or Rejected.

Applicant	Monthly Income	Credit Score	Govt Job	Existing Loan	Past Default	Class
A1	25000	580	0	1	1	Rejected
A2	30000	600	0	1	0	Rejected
A3	60000	750	1	0	0	Approved
A4	70000	800	1	0	0	Approved
A5	40000	650	0	1	0	Rejected
A6	80000	820	1	0	0	Approved

A new applicant has:

Monthly Income	Credit Score	Govt Job	Existing Loan	Past Default
55000	700	1	0	0

Using KNN with K = 3, classify the loan application as Approved or Rejected using all four distance measures.

ANSWER

(a) Features used

Numerical features: Monthly Income, Credit Score

Binary features: Govt Job, Existing Loan, Past Default

(b-e) Distance calculation (new sample = [55000, 700, 1, 0, 0])

Sample calculation for A1 (the same formulas are applied to every training sample):

```
Euclidean(A1) = sqrt((55000-25000)^2 + (700-580)^2 + (1-0)^2 + (0-1)^2 + (0-1)^2) = sqrt(9e+08 + 14400 + 1 + 1 + 1)
= sqrt(9.00014e+08) = 30000.240
Manhattan(A1) = |55000-25000| + |700-580| + |1-0| + |0-1| + |0-1| = 30000 + 120 + 1 + 1 + 1 = 30123
Minkowski(A1,p=3) = (|55000-25000|^3 + |700-580|^3 + |1-0|^3 + |0-1|^3 + |0-1|^3)^(1/3) = (2.7e+13 + 1.728e+06 + 1 +
1 + 1)^(1/3) = 30000.001
Hamming(A1) = compare binary new(1, 0, 0) vs A1(0, 1, 1) -> 3 mismatch(es) = 3
```

Distances from the new sample to all training samples:

Sample	Class	Euclidean (b)	Manhattan (c)	Minkowski p=3 (d)	Hamming (e)
A1	Rejected	30000.240	30123	30000.001	3
A2	Rejected	25000.200	25102	25000.001	2
A3	Approved	5000.250	5050	5000.002	0
A4	Approved	15000.333	15100	15000.001	0
A5	Rejected	15000.083	15052	15000.000	2
A6	Approved	25000.288	25120	25000.001	0

(f) 3 Nearest Neighbours & (g) Majority-vote Prediction

Distance measure	3 Nearest Neighbours	Votes	Prediction
Euclidean	A3 (Approved), A5 (Rejected), A4 (Approved)	Approved:2; Rejected:1	Approved
Manhattan	A3 (Approved), A5 (Rejected), A4 (Approved)	Approved:2; Rejected:1	Approved
Minkowski p=3	A3 (Approved), A5 (Rejected), A4 (Approved)	Approved:2; Rejected:1	Approved
Hamming	A3 (Approved), A4 (Approved), A6 (Approved)	Approved:3	Approved

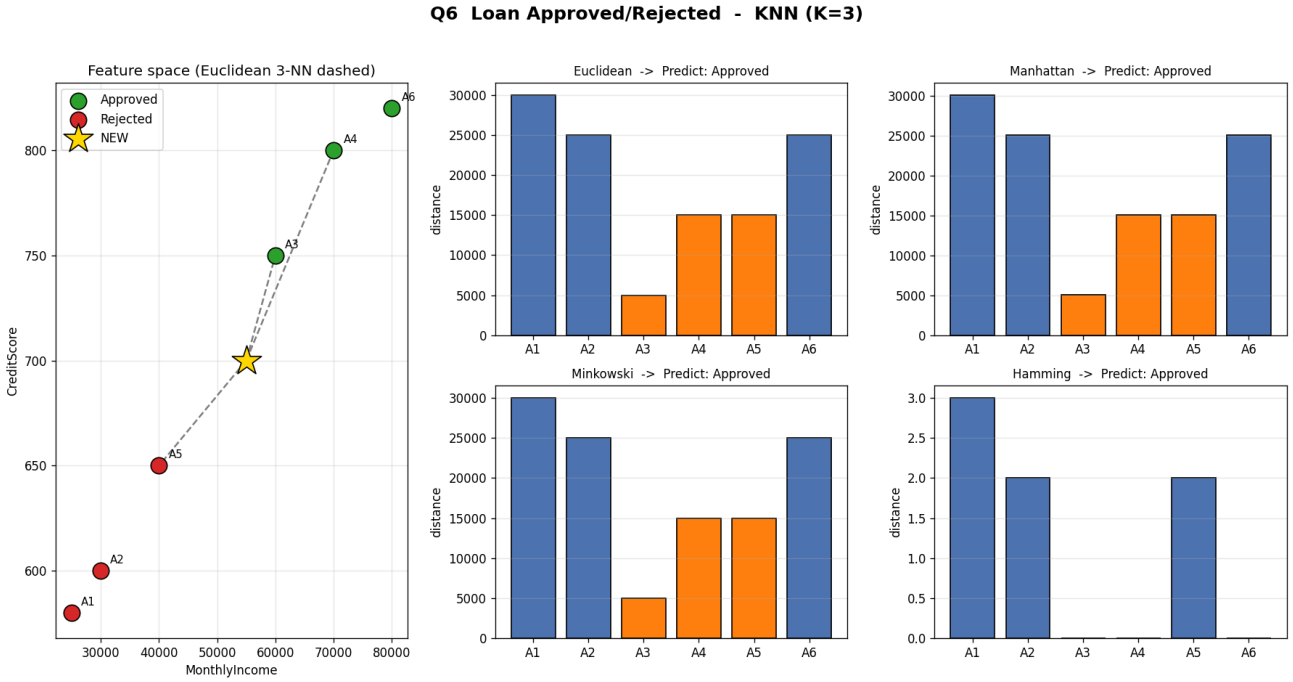
Python Code

```
"""
Question 6 : Loan Approval Prediction (Approved / Rejected)  --  KNN (K = 3)
```

```
--------------------------------------------------------------------------------
Input  : q6_loan.csv  (6 training rows + 1 NEW row)
Output : console results + a graph (q6_loan_knn.png)

Distance measures : Euclidean, Manhattan, Minkowski(p=3), Hamming
  * Euclidean / Manhattan / Minkowski  -> use ALL 5 features
  * Hamming                            -> use only the 3 BINARY features
"""


import numpy as np
import pandas as pd
import matplotlib.pyplot as plt


# ---------------------------------------------------------------- configuration
CSV_FILE   = "q6_loan.csv"
NUMERIC    = ["MonthlyIncome", "CreditScore"]
BINARY     = ["GovtJob", "ExistingLoan", "PastDefault"]
LABEL_COL  = "Class"
K          = 3
P_MINKOWSKI = 3
TITLE      = "Q6  Loan Approved/Rejected"
PNG_FILE   = "q6_loan_knn.png"


# ---------------------------------------------------------------- read csv input
df = pd.read_csv(CSV_FILE)
train = df[df["ID"] != "NEW"].reset_index(drop=True)
new   = df[df["ID"] == "NEW"].reset_index(drop=True)

ALL_FEATURES = NUMERIC + BINARY

print("(a) Numerical features :", NUMERIC)
print("    Binary   features  :", BINARY)
print("\nTraining data:\n", train[["ID"] + ALL_FEATURES + [LABEL_COL]].to_string(index=False))
print("\nNew sample:\n", new[["ID"] + ALL_FEATURES].to_string(index=False), "\n")


# ---------------------------------------------------------------- distances
X  = train[ALL_FEATURES].to_numpy(dtype=float)
q  = new[ALL_FEATURES].to_numpy(dtype=float)[0]
Xb = train[BINARY].to_numpy(dtype=int)
qb = new[BINARY].to_numpy(dtype=int)[0]

diff = X - q
euclidean = np.sqrt((diff ** 2).sum(axis=1))                      # (b)
manhattan = np.abs(diff).sum(axis=1)                             # (c)
minkowski = (np.abs(diff) ** P_MINKOWSKI).sum(axis=1) ** (1 / P_MINKOWSKI)  # (d)
hamming   = (Xb != qb).sum(axis=1)                               # (e)

results = pd.DataFrame({
    "ID": train["ID"],
    LABEL_COL: train[LABEL_COL],
    "Euclidean": euclidean.round(3),
    "Manhattan": manhattan.round(3),
    "Minkowski": minkowski.round(3),
    "Hamming": hamming,
})
print("(b-e) Distances to every training sample:\n", results.to_string(index=False), "\n")


# ---------------------------------------------------------------- 3-NN + voting
measures = {"Euclidean": euclidean, "Manhattan": manhattan,
            "Minkowski": minkowski, "Hamming": hamming}
predictions = {}

for name, dist in measures.items():
    order = np.argsort(dist, kind="stable")[:K]          # (f)
    neighbours = train.loc[order, "ID"].tolist()
    votes = train.loc[order, LABEL_COL].value_counts()
    pred = votes.idxmax()                                # (g)
    predictions[name] = pred
    print(f"{name:10s} -> 3 nearest: {neighbours}  votes: {dict(votes)}  => {pred}")

print("\n(g) Predictions:", predictions)
```

```
# ------------------------------------------------------------------ graphs
classes = sorted(train[LABEL_COL].unique())
palette = {classes[0]: "#2ca02c", classes[1]: "#d62728"}   # Approved green, Rejected red

fig = plt.figure(figsize=(15, 8))
fig.suptitle(TITLE + "  -  KNN (K=3)", fontsize=15, fontweight="bold")
gs = fig.add_gridspec(2, 3)

ax = fig.add_subplot(gs[:, 0])
for cls in classes:
    sub = train[train[LABEL_COL] == cls]
    ax.scatter(sub[NUMERIC[0]], sub[NUMERIC[1]], s=180, c=palette[cls],
               label=cls, edgecolors="black", zorder=3)
for _, r in train.iterrows():
    ax.annotate(r["ID"], (r[NUMERIC[0]], r[NUMERIC[1]]),
                textcoords="offset points", xytext=(8, 6), fontsize=9)
ax.scatter(new[NUMERIC[0]], new[NUMERIC[1]], marker="*", s=650, c="gold",
           edgecolors="black", label="NEW", zorder=4)
for idx in np.argsort(euclidean)[:K]:
    ax.plot([new[NUMERIC[0]].iloc[0], train[NUMERIC[0]].iloc[idx]],
            [new[NUMERIC[1]].iloc[0], train[NUMERIC[1]].iloc[idx]],
            "--", c="gray", zorder=2)
ax.set_xlabel(NUMERIC[0]); ax.set_ylabel(NUMERIC[1])
ax.set_title("Feature space (Euclidean 3-NN dashed)")
ax.legend(); ax.grid(alpha=0.3)

positions = [gs[0, 1], gs[0, 2], gs[1, 1], gs[1, 2]]
for (name, dist), pos in zip(measures.items(), positions):
    axb = fig.add_subplot(pos)
    nearest = set(np.argsort(dist, kind="stable")[:K])
    colors = ["#ff7f0e" if i in nearest else "#4c72b0" for i in range(len(dist))]
    axb.bar(train["ID"], dist, color=colors, edgecolor="black")
    axb.set_title(f"{name}  ->  Predict: {predictions[name]}", fontsize=10)
    axb.set_ylabel("distance"); axb.grid(axis="y", alpha=0.3)

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.savefig(PNG_FILE, dpi=120)
print("\nGraph saved as", PNG_FILE)
plt.show()
```

Graph Output



(h) Comparison & Final Conclusion

- All four measures give APPROVED.

- Income (tens of thousands) dominates the geometric distances and places the applicant nearest A3 (60k) and A4 (70k).

- Hamming is 0 to every Approved record (govt job, no existing loan, no default), confirming the same class.

- Most suitable: Euclidean - the dominant numeric feature (income) already aligns with the correct class; ideally standardise income & credit score.

Final Answer (majority vote): APPROVED

Question 7: Movie Preference Classification

A movie platform wants to classify a user as an Action Lover or Comedy Lover.

User	Action Rating	Comedy Rating	Watched Superhero Movie	Watched Stand-up Show	Watched Sitcom	Class
U1	5	1	1	0	0	Action Lover
U2	4	2	1	0	0	Action Lover
U3	1	5	0	1	1	Comedy Lover
U4	2	4	0	1	1	Comedy Lover
U5	5	2	1	0	0	Action Lover
U6	1	4	0	1	1	Comedy Lover

A new user has:

Action Rating	Comedy Rating	Watched Superhero Movie	Watched Stand-up Show	Watched Sitcom
3	3	1	1	0

Using KNN with K = 3, classify the new user as Action Lover or Comedy Lover using Euclidean, Manhattan, Minkowski, and Hamming distance.

ANSWER

(a) Features used

Numerical features: Action Rating, Comedy Rating

Binary features: Watched Superhero Movie, Watched Stand-up Show, Watched Sitcom

(b-e) Distance calculation (new sample = [3, 3, 1, 1, 0])

Sample calculation for U1 (the same formulas are applied to every training sample):

```
Euclidean(U1) = sqrt((3-5)^2 + (3-1)^2 + (1-1)^2 + (1-0)^2 + (0-0)^2) = sqrt(4 + 4 + 0 + 1 + 0) = sqrt(9) = 3.000
Manhattan(U1) = |3-5| + |3-1| + |1-1| + |1-0| + |0-0| = 2 + 2 + 0 + 1 + 0 = 5
Minkowski(U1,p=3) = (|3-5|^3 + |3-1|^3 + |1-1|^3 + |1-0|^3 + |0-0|^3)^(1/3) = (8 + 8 + 0 + 1 + 0)^(1/3) = 2.571
Hamming(U1) = compare binary new(1, 1, 0) vs U1(1, 0, 0) -> 1 mismatch(es) = 1
```

Distances from the new sample to all training samples:

Sample	Class	Euclidean (b)	Manhattan (c)	Minkowski p=3 (d)	Hamming (e)
U1	Action Lover	3.000	5	2.571	1
U2	Action Lover	1.732	3	1.442	1
U3	Comedy Lover	3.162	6	2.621	2
U4	Comedy Lover	2.000	4	1.587	2
U5	Action Lover	2.449	4	2.154	1
U6	Comedy Lover	2.646	5	2.224	2

(f) 3 Nearest Neighbours & (g) Majority-vote Prediction

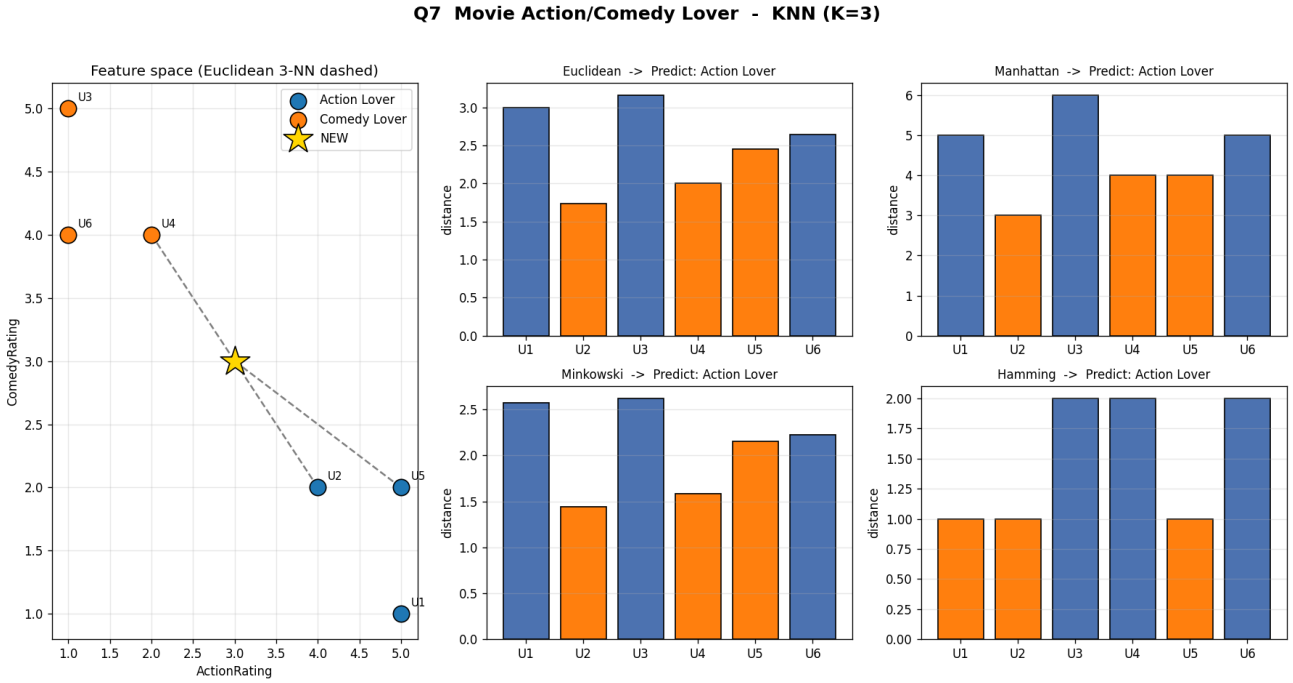
Distance measure	3 Nearest Neighbours	Votes	Prediction
Euclidean	U2 (Action Lover), U4 (Comedy Lover), U5 (Action Lover)	Action Lover:2; Comedy Lover:1	Action Lover
Manhattan	U2 (Action Lover), U4 (Comedy Lover), U5 (Action Lover)	Action Lover:2; Comedy Lover:1	Action Lover
Minkowski p=3	U2 (Action Lover), U4 (Comedy Lover), U5 (Action Lover)	Action Lover:2; Comedy Lover:1	Action Lover
Hamming	U1 (Action Lover), U2 (Action Lover), U5 (Action Lover)	Action Lover:3	Action Lover

Python Code

```
"""
Question 7 : Movie Preference (Action Lover / Comedy Lover)  --  KNN (K = 3)
-------------------------------------------------------------------------------
Input  : q7_movie.csv  (6 training rows + 1 NEW row)
Output : console results + a graph (q7_movie_knn.png)

Distance measures : Euclidean, Manhattan, Minkowski(p=3), Hamming
  * Euclidean / Manhattan / Minkowski  -> use ALL 5 features
  * Hamming                            -> use only the 3 BINARY features
"""


import numpy as np
import pandas as pd
import matplotlib.pyplot as plt


# ---------------------------------------------------------------- configuration
CSV_FILE   = "q7_movie.csv"
NUMERIC    = ["ActionRating", "ComedyRating"]
BINARY     = ["WatchedSuperhero", "WatchedStandup", "WatchedSitcom"]
LABEL_COL  = "Class"
K          = 3
P_MINKOWSKI = 3
TITLE      = "Q7  Movie Action/Comedy Lover"
PNG_FILE   = "q7_movie_knn.png"


# ---------------------------------------------------------------- read csv input
df = pd.read_csv(CSV_FILE)
train = df[df["ID"] != "NEW"].reset_index(drop=True)
new   = df[df["ID"] == "NEW"].reset_index(drop=True)


ALL_FEATURES = NUMERIC + BINARY

print("(a) Numerical features :", NUMERIC)
print("    Binary   features  :", BINARY)
print("\nTraining data:\n", train[["ID"] + ALL_FEATURES + [LABEL_COL]].to_string(index=False))
print("\nNew sample:\n", new[["ID"] + ALL_FEATURES].to_string(index=False), "\n")


# ---------------------------------------------------------------- distances
X  = train[ALL_FEATURES].to_numpy(dtype=float)
q  = new[ALL_FEATURES].to_numpy(dtype=float)[0]
Xb = train[BINARY].to_numpy(dtype=int)
qb = new[BINARY].to_numpy(dtype=int)[0]

diff = X - q
euclidean = np.sqrt((diff ** 2).sum(axis=1))                      # (b)
manhattan = np.abs(diff).sum(axis=1)                             # (c)
minkowski = (np.abs(diff) ** P_MINKOWSKI).sum(axis=1) ** (1 / P_MINKOWSKI)  # (d)
hamming   = (Xb != qb).sum(axis=1)                                # (e)

results = pd.DataFrame({
    "ID": train["ID"],
    LABEL_COL: train[LABEL_COL],
    "Euclidean": euclidean.round(3),
    "Manhattan": manhattan.round(3),
    "Minkowski": minkowski.round(3),
    "Hamming": hamming,
})
print("(b-e) Distances to every training sample:\n", results.to_string(index=False), "\n")


# ---------------------------------------------------------------- 3-NN + voting
measures = {"Euclidean": euclidean, "Manhattan": manhattan,
            "Minkowski": minkowski, "Hamming": hamming}
predictions = {}

for name, dist in measures.items():
    order = np.argsort(dist, kind="stable")[:K]          # (f)
    neighbours = train.loc[order, "ID"].tolist()
    votes = train.loc[order, LABEL_COL].value_counts()
    pred = votes.idxmax()                                # (g)
    predictions[name] = pred
    print(f"{name:10s} -> 3 nearest: {neighbours}  votes: {dict(votes)}  => {pred}")


print("\n(g) Predictions:", predictions)
```

```
# ------------------------------------------------------------------ graphs
classes = sorted(train[LABEL_COL].unique())
palette = {classes[0]: "#1f77b4", classes[1]: "#ff7f0e"}   # Action blue, Comedy orange

fig = plt.figure(figsize=(15, 8))
fig.suptitle(TITLE + "  -  KNN (K=3)", fontsize=15, fontweight="bold")
gs = fig.add_gridspec(2, 3)

ax = fig.add_subplot(gs[:, 0])
for cls in classes:
    sub = train[train[LABEL_COL] == cls]
    ax.scatter(sub[NUMERIC[0]], sub[NUMERIC[1]], s=180, c=palette[cls],
               label=cls, edgecolors="black", zorder=3)
for _, r in train.iterrows():
    ax.annotate(r["ID"], (r[NUMERIC[0]], r[NUMERIC[1]]),
                textcoords="offset points", xytext=(8, 6), fontsize=9)
ax.scatter(new[NUMERIC[0]], new[NUMERIC[1]], marker="*", s=650, c="gold",
           edgecolors="black", label="NEW", zorder=4)
for idx in np.argsort(euclidean)[:K]:
    ax.plot([new[NUMERIC[0]].iloc[0], train[NUMERIC[0]].iloc[idx]],
            [new[NUMERIC[1]].iloc[0], train[NUMERIC[1]].iloc[idx]],
            "--", c="gray", zorder=2)
ax.set_xlabel(NUMERIC[0]); ax.set_ylabel(NUMERIC[1])
ax.set_title("Feature space (Euclidean 3-NN dashed)")
ax.legend(); ax.grid(alpha=0.3)

positions = [gs[0, 1], gs[0, 2], gs[1, 1], gs[1, 2]]
for (name, dist), pos in zip(measures.items(), positions):
    axb = fig.add_subplot(pos)
    nearest = set(np.argsort(dist, kind="stable")[:K])
    colors = ["#ff7f0e" if i in nearest else "#4c72b0" for i in range(len(dist))]
    axb.bar(train["ID"], dist, color=colors, edgecolor="black")
    axb.set_title(f"{name}  ->  Predict: {predictions[name]}", fontsize=10)
    axb.set_ylabel("distance"); axb.grid(axis="y", alpha=0.3)

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.savefig(PNG_FILE, dpi=120)
print("\nGraph saved as", PNG_FILE)
plt.show()
```

Graph Output



(h) Comparison & Final Conclusion

- All four measures give ACTION LOVER.
- Ratings are tied (3,3) so the numeric features are neutral; the binary viewing history breaks the tie.
- The user watched a superhero movie (Action marker) and Hamming = 1 to the Action users U1, U2, U5.
- Most suitable: because the numeric ratings are neutral, the binary behaviour (Hamming) is the deciding signal.

Final Answer (majority vote): ACTION LOVER

Question 8: Crop Disease Detection

An agriculture research center wants to classify a plant as Healthy or Diseased.

Plant	Leaf Spot Count	Leaf Color Index	Yellow Leaves	White Fungus	Wilting	Class
L1	2	80	0	0	0	Healthy
L2	3	75	0	0	0	Healthy
L3	12	40	1	1	1	Diseased
L4	15	35	1	1	1	Diseased
L5	5	70	0	0	1	Healthy
L6	14	38	1	1	1	Diseased

A new plant has:

Leaf Spot Count	Leaf Color Index	Yellow Leaves	White Fungus	Wilting
10	50	1	1	0

Using KNN with K = 3, classify the plant as Healthy or Diseased using all four distance measures.

ANSWER

(a) Features used

Numerical features: Leaf Spot Count, Leaf Color Index

Binary features: Yellow Leaves, White Fungus, Wilting

(b-e) Distance calculation (new sample = [10, 50, 1, 1, 0])

Sample calculation for L1 (the same formulas are applied to every training sample):

```
Euclidean(L1) = sqrt((10-2)^2 + (50-80)^2 + (1-0)^2 + (1-0)^2 + (0-0)^2) = sqrt(64 + 900 + 1 + 1 + 0) = sqrt(966) = 31.081
Manhattan(L1) = |10-2| + |50-80| + |1-0| + |1-0| + |0-0| = 8 + 30 + 1 + 1 + 0 = 40
Minkowski(L1,p=3) = (|10-2|^3 + |50-80|^3 + |1-0|^3 + |1-0|^3 + |0-0|^3)^(1/3) = (512 + 27000 + 1 + 1 + 0)^(1/3) = 30.189
Hamming(L1) = compare binary new(1, 1, 0) vs L1(0, 0, 0) -> 2 mismatch(es) = 2
```

Distances from the new sample to all training samples:

Sample	Class	Euclidean (b)	Manhattan (c)	Minkowski p=3 (d)	Hamming (e)
L1	Healthy	31.081	40	30.189	2
L2	Healthy	26.000	34	25.183	2
L3	Diseased	10.247	13	10.030	1
L4	Diseased	15.843	21	15.184	1
L5	Healthy	20.688	28	20.106	3
L6	Diseased	12.689	17	12.149	1

(f) 3 Nearest Neighbours & (g) Majority-vote Prediction

Distance measure	3 Nearest Neighbours	Votes	Prediction
Euclidean	L3 (Diseased), L6 (Diseased), L4 (Diseased)	Diseased:3	Diseased
Manhattan	L3 (Diseased), L6 (Diseased), L4 (Diseased)	Diseased:3	Diseased
Minkowski p=3	L3 (Diseased), L6 (Diseased), L4 (Diseased)	Diseased:3	Diseased
Hamming	L3 (Diseased), L4 (Diseased), L6 (Diseased)	Diseased:3	Diseased

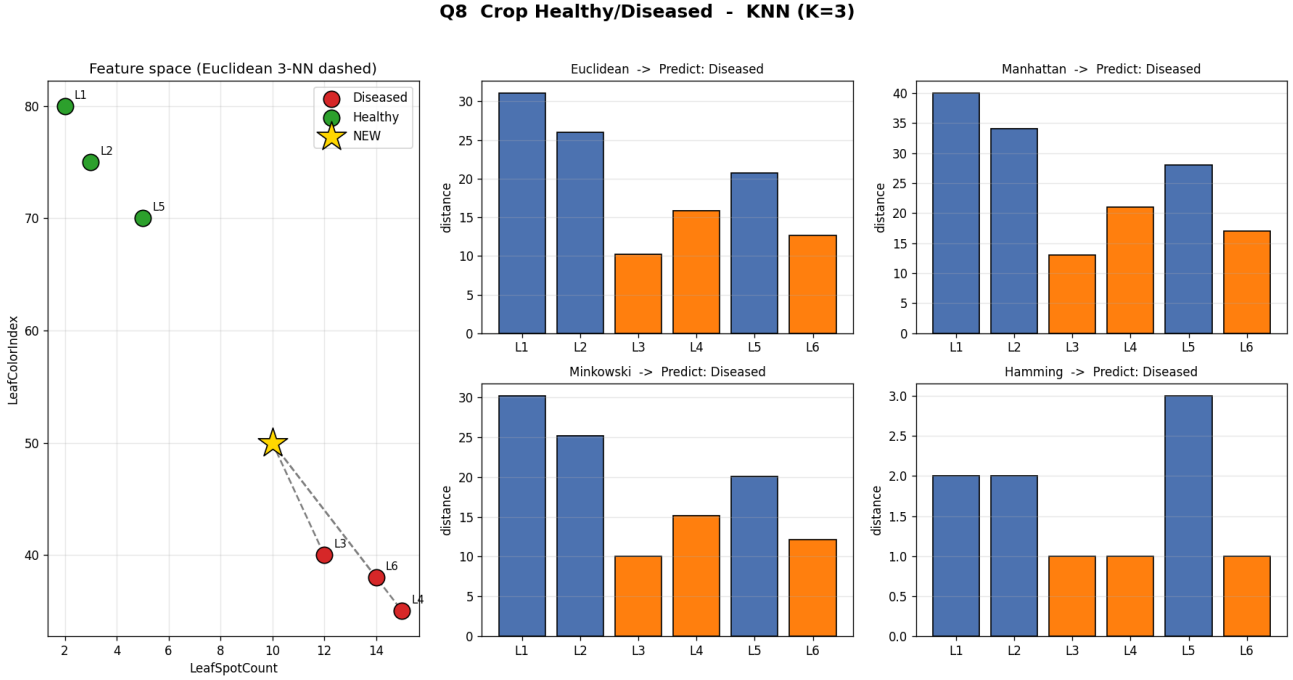
Python Code

```
"""
Question 8 : Crop Disease Detection (Healthy / Diseased)  --  KNN (K = 3)
```

```
------------------------------------------------------------------------
Input  : q8_crop.csv  (6 training rows + 1 NEW row)
Output : console results + a graph (q8_crop_knn.png)

Distance measures : Euclidean, Manhattan, Minkowski(p=3), Hamming
  * Euclidean / Manhattan / Minkowski  -> use ALL 5 features
  * Hamming                            -> use only the 3 BINARY features
"""


import numpy as np
import pandas as pd
import matplotlib.pyplot as plt


# ---------------------------------------------------------------- configuration
CSV_FILE   = "q8_crop.csv"
NUMERIC    = ["LeafSpotCount", "LeafColorIndex"]
BINARY     = ["YellowLeaves", "WhiteFungus", "Wilting"]
LABEL_COL  = "Class"
K          = 3
P_MINKOWSKI = 3
TITLE      = "Q8  Crop Healthy/Diseased"
PNG_FILE   = "q8_crop_knn.png"


# ---------------------------------------------------------------- read csv input
df = pd.read_csv(CSV_FILE)
train = df[df["ID"] != "NEW"].reset_index(drop=True)
new   = df[df["ID"] == "NEW"].reset_index(drop=True)

ALL_FEATURES = NUMERIC + BINARY

print("(a) Numerical features :", NUMERIC)
print("    Binary   features  :", BINARY)
print("\nTraining data:\n", train[["ID"] + ALL_FEATURES + [LABEL_COL]].to_string(index=False))
print("\nNew sample:\n", new[["ID"] + ALL_FEATURES].to_string(index=False), "\n")


# ---------------------------------------------------------------- distances
X  = train[ALL_FEATURES].to_numpy(dtype=float)
q  = new[ALL_FEATURES].to_numpy(dtype=float)[0]
Xb = train[BINARY].to_numpy(dtype=int)
qb = new[BINARY].to_numpy(dtype=int)[0]

diff = X - q
euclidean = np.sqrt((diff ** 2).sum(axis=1))                      # (b)
manhattan = np.abs(diff).sum(axis=1)                             # (c)
minkowski = (np.abs(diff) ** P_MINKOWSKI).sum(axis=1) ** (1 / P_MINKOWSKI)  # (d)
hamming   = (Xb != qb).sum(axis=1)                               # (e)

results = pd.DataFrame({
    "ID": train["ID"],
    LABEL_COL: train[LABEL_COL],
    "Euclidean": euclidean.round(3),
    "Manhattan": manhattan.round(3),
    "Minkowski": minkowski.round(3),
    "Hamming": hamming,
})
print("(b-e) Distances to every training sample:\n", results.to_string(index=False), "\n")


# ---------------------------------------------------------------- 3-NN + voting
measures = {"Euclidean": euclidean, "Manhattan": manhattan,
            "Minkowski": minkowski, "Hamming": hamming}
predictions = {}

for name, dist in measures.items():
    order = np.argsort(dist, kind="stable")[:K]          # (f)
    neighbours = train.loc[order, "ID"].tolist()
    votes = train.loc[order, LABEL_COL].value_counts()
    pred = votes.idxmax()                                # (g)
    predictions[name] = pred
    print(f"{name:10s} -> 3 nearest: {neighbours}  votes: {dict(votes)}  => {pred}")

print("\n(g) Predictions:", predictions)
```

```
# ------------------------------------------------------------------ graphs
classes = sorted(train[LABEL_COL].unique())
palette = {classes[0]: "#d62728", classes[1]: "#2ca02c"}   # Diseased red, Healthy green

fig = plt.figure(figsize=(15, 8))
fig.suptitle(TITLE + "  -  KNN (K=3)", fontsize=15, fontweight="bold")
gs = fig.add_gridspec(2, 3)

ax = fig.add_subplot(gs[:, 0])
for cls in classes:
    sub = train[train[LABEL_COL] == cls]
    ax.scatter(sub[NUMERIC[0]], sub[NUMERIC[1]], s=180, c=palette[cls],
               label=cls, edgecolors="black", zorder=3)
for _, r in train.iterrows():
    ax.annotate(r["ID"], (r[NUMERIC[0]], r[NUMERIC[1]]),
                textcoords="offset points", xytext=(8, 6), fontsize=9)
ax.scatter(new[NUMERIC[0]], new[NUMERIC[1]], marker="*", s=650, c="gold",
           edgecolors="black", label="NEW", zorder=4)
for idx in np.argsort(euclidean)[:K]:
    ax.plot([new[NUMERIC[0]].iloc[0], train[NUMERIC[0]].iloc[idx]],
            [new[NUMERIC[1]].iloc[0], train[NUMERIC[1]].iloc[idx]],
            "--", c="gray", zorder=2)
ax.set_xlabel(NUMERIC[0]); ax.set_ylabel(NUMERIC[1])
ax.set_title("Feature space (Euclidean 3-NN dashed)")
ax.legend(); ax.grid(alpha=0.3)

positions = [gs[0, 1], gs[0, 2], gs[1, 1], gs[1, 2]]
for (name, dist), pos in zip(measures.items(), positions):
    axb = fig.add_subplot(pos)
    nearest = set(np.argsort(dist, kind="stable")[:K])
    colors = ["#ff7f0e" if i in nearest else "#4c72b0" for i in range(len(dist))]
    axb.bar(train["ID"], dist, color=colors, edgecolor="black")
    axb.set_title(f"{name}  ->  Predict: {predictions[name]}", fontsize=10)
    axb.set_ylabel("distance"); axb.grid(axis="y", alpha=0.3)

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.savefig(PNG_FILE, dpi=120)
print("\nGraph saved as", PNG_FILE)
plt.show()
```

Graph Output



(h) Comparison & Final Conclusion

- All four measures give DISEASED - unanimous.

- High leaf-spot count and low colour index sit inside the Diseased cluster (L3, L4, L6); yellowing + fungus reinforce it.

- Numeric symptoms and binary symptoms agree, so every metric gives the same answer.

- Most suitable: Euclidean - it captures the combined severity of spot count and colour degradation.

Final Answer (majority vote): DISEASED

Question 9: Network Intrusion Detection

A cybersecurity system wants to classify network activity as Normal or Attack.

Record	Packets per Second	Failed Login Attempts	Unknown Port Used	Admin Access Tried	Blacklisted IP	Class
N1	20	0	0	0	0	Normal
N2	30	1	0	0	0	Normal
N3	100	5	1	1	1	Attack
N4	120	6	1	1	1	Attack
N5	40	1	0	0	0	Normal
N6	110	7	1	1	1	Attack

A new network record has:

Packets per Second	Failed Login Attempts	Unknown Port Used	Admin Access Tried	Blacklisted IP
90	4	1	1	0

Using KNN with K = 3, classify the network activity as Normal or Attack using Euclidean, Manhattan, Minkowski, and Hamming distance.

ANSWER

(a) Features used

Numerical features: Packets per Second, Failed Login Attempts

Binary features: Unknown Port Used, Admin Access Tried, Blacklisted IP

(b-e) Distance calculation (new sample = [90, 4, 1, 1, 0])

Sample calculation for N1 (the same formulas are applied to every training sample):

```
Euclidean(N1) = sqrt((90-20)^2 + (4-0)^2 + (1-0)^2 + (1-0)^2 + (0-0)^2) = sqrt(4900 + 16 + 1 + 1 + 0) = sqrt(4918) = 70.128
Manhattan(N1) = |90-20| + |4-0| + |1-0| + |1-0| + |0-0| = 70 + 4 + 1 + 1 + 0 = 76
Minkowski(N1,p=3) = (|90-20|^3 + |4-0|^3 + |1-0|^3 + |1-0|^3 + |0-0|^3)^(1/3) = (343000 + 64 + 1 + 1 + 0)^(1/3) = 70.004
Hamming(N1) = compare binary new(1, 1, 0) vs N1(0, 0, 0) -> 2 mismatch(es) = 2
```

Distances from the new sample to all training samples:

Sample	Class	Euclidean (b)	Manhattan (c)	Minkowski p=3 (d)	Hamming (e)
N1	Normal	70.128	76	70.004	2
N2	Normal	60.092	65	60.003	2
N3	Attack	10.100	12	10.007	1
N4	Attack	30.083	33	30.003	1
N5	Normal	50.110	55	50.004	2
N6	Attack	20.248	24	20.023	1

(f) 3 Nearest Neighbours & (g) Majority-vote Prediction

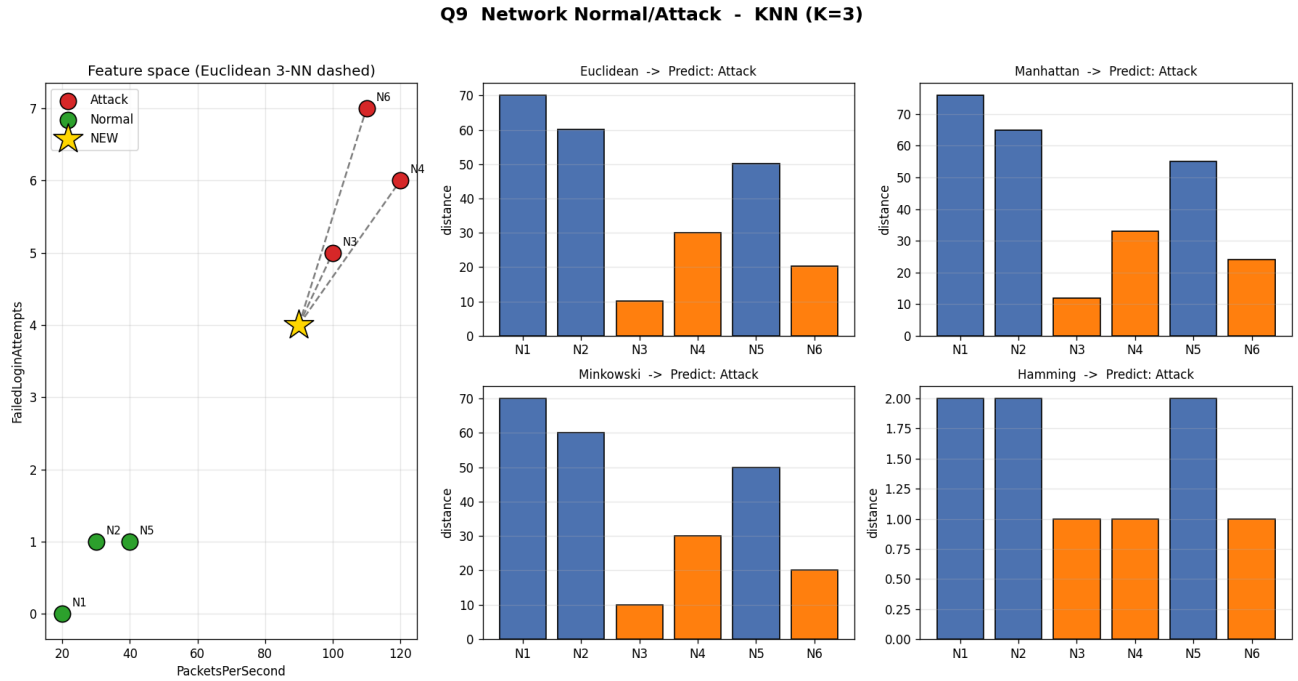
Distance measure	3 Nearest Neighbours	Votes	Prediction
Euclidean	N3 (Attack), N6 (Attack), N4 (Attack)	Attack:3	Attack
Manhattan	N3 (Attack), N6 (Attack), N4 (Attack)	Attack:3	Attack
Minkowski p=3	N3 (Attack), N6 (Attack), N4 (Attack)	Attack:3	Attack
Hamming	N3 (Attack), N4 (Attack), N6 (Attack)	Attack:3	Attack

Python Code

```
"""
Question 9 : Network Intrusion Detection (Normal / Attack)  --  KNN (K = 3)
--------------------------------------------------------------------------
Input  : q9_network.csv  (6 training rows + 1 NEW row)
Output : console results + a graph (q9_network_knn.png)

Distance measures : Euclidean, Manhattan, Minkowski(p=3), Hamming
  * Euclidean / Manhattan / Minkowski  -> use ALL 5 features
  * Hamming                            -> use only the 3 BINARY features
"""


import numpy as np
import pandas as pd
import matplotlib.pyplot as plt


# ---------------------------------------------------------------- configuration
CSV_FILE   = "q9_network.csv"
NUMERIC    = ["PacketsPerSecond", "FailedLoginAttempts"]
BINARY     = ["UnknownPortUsed", "AdminAccessTried", "BlacklistedIP"]
LABEL_COL  = "Class"
K          = 3
P_MINKOWSKI = 3
TITLE      = "Q9  Network Normal/Attack"
PNG_FILE   = "q9_network_knn.png"


# ---------------------------------------------------------------- read csv input
df = pd.read_csv(CSV_FILE)
train = df[df["ID"] != "NEW"].reset_index(drop=True)
new   = df[df["ID"] == "NEW"].reset_index(drop=True)


ALL_FEATURES = NUMERIC + BINARY

print("(a) Numerical features :", NUMERIC)
print("    Binary   features  :", BINARY)
print("\nTraining data:\n", train[["ID"] + ALL_FEATURES + [LABEL_COL]].to_string(index=False))
print("\nNew sample:\n", new[["ID"] + ALL_FEATURES].to_string(index=False), "\n")


# ---------------------------------------------------------------- distances
X  = train[ALL_FEATURES].to_numpy(dtype=float)
q  = new[ALL_FEATURES].to_numpy(dtype=float)[0]
Xb = train[BINARY].to_numpy(dtype=int)
qb = new[BINARY].to_numpy(dtype=int)[0]

diff = X - q
euclidean = np.sqrt((diff ** 2).sum(axis=1))                      # (b)
manhattan = np.abs(diff).sum(axis=1)                             # (c)
minkowski = (np.abs(diff) ** P_MINKOWSKI).sum(axis=1) ** (1 / P_MINKOWSKI)  # (d)
hamming   = (Xb != qb).sum(axis=1)                                # (e)

results = pd.DataFrame({
    "ID": train["ID"],
    LABEL_COL: train[LABEL_COL],
    "Euclidean": euclidean.round(3),
    "Manhattan": manhattan.round(3),
    "Minkowski": minkowski.round(3),
    "Hamming": hamming,
})
print("(b-e) Distances to every training sample:\n", results.to_string(index=False), "\n")


# ---------------------------------------------------------------- 3-NN + voting
measures = {"Euclidean": euclidean, "Manhattan": manhattan,
            "Minkowski": minkowski, "Hamming": hamming}
predictions = {}

for name, dist in measures.items():
    order = np.argsort(dist, kind="stable")[:K]          # (f)
    neighbours = train.loc[order, "ID"].tolist()
    votes = train.loc[order, LABEL_COL].value_counts()
    pred = votes.idxmax()                                 # (g)
    predictions[name] = pred
    print(f"{name:10s} -> 3 nearest: {neighbours}  votes: {dict(votes)}  => {pred}")


print("\n(g) Predictions:", predictions)
```

```
# ------------------------------------------------------------------ graphs
classes = sorted(train[LABEL_COL].unique())
palette = {classes[0]: "#d62728", classes[1]: "#2ca02c"}   # Attack red, Normal green

fig = plt.figure(figsize=(15, 8))
fig.suptitle(TITLE + "  -  KNN (K=3)", fontsize=15, fontweight="bold")
gs = fig.add_gridspec(2, 3)

ax = fig.add_subplot(gs[:, 0])
for cls in classes:
    sub = train[train[LABEL_COL] == cls]
    ax.scatter(sub[NUMERIC[0]], sub[NUMERIC[1]], s=180, c=palette[cls],
               label=cls, edgecolors="black", zorder=3)
for _, r in train.iterrows():
    ax.annotate(r["ID"], (r[NUMERIC[0]], r[NUMERIC[1]]),
                textcoords="offset points", xytext=(8, 6), fontsize=9)
ax.scatter(new[NUMERIC[0]], new[NUMERIC[1]], marker="*", s=650, c="gold",
           edgecolors="black", label="NEW", zorder=4)
for idx in np.argsort(euclidean)[:K]:
    ax.plot([new[NUMERIC[0]].iloc[0], train[NUMERIC[0]].iloc[idx]],
            [new[NUMERIC[1]].iloc[0], train[NUMERIC[1]].iloc[idx]],
            "--", c="gray", zorder=2)
ax.set_xlabel(NUMERIC[0]); ax.set_ylabel(NUMERIC[1])
ax.set_title("Feature space (Euclidean 3-NN dashed)")
ax.legend(); ax.grid(alpha=0.3)

positions = [gs[0, 1], gs[0, 2], gs[1, 1], gs[1, 2]]
for (name, dist), pos in zip(measures.items(), positions):
    axb = fig.add_subplot(pos)
    nearest = set(np.argsort(dist, kind="stable")[:K])
    colors = ["#ff7f0e" if i in nearest else "#4c72b0" for i in range(len(dist))]
    axb.bar(train["ID"], dist, color=colors, edgecolor="black")
    axb.set_title(f"{name}  ->  Predict: {predictions[name]}", fontsize=10)
    axb.set_ylabel("distance"); axb.grid(axis="y", alpha=0.3)

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.savefig(PNG_FILE, dpi=120)
print("\nGraph saved as", PNG_FILE)
plt.show()
```

Graph Output



(h) Comparison & Final Conclusion

- All four measures give ATTACK - unanimous.

- High packet rate + failed logins place the record next to the Attack cluster (N3, N4, N6); unknown port + admin attempt confirm it.

- Both the dominant numeric feature (packets/s) and the binary intrusion flags point to the same class.

- Most suitable: Euclidean - the packet-rate gradient cleanly separates Normal from Attack traffic.

Final Answer (majority vote): ATTACK

Question 10: Employee Attrition Prediction

A company wants to predict whether an employee is likely to Stay or Leave.

Employee	Experience in Years	Satisfaction Score	Overtime	Promotion Received	Workplace Conflict	Class
E1	2	8	0	1	0	Stay
E2	3	7	0	1	0	Stay
E3	6	3	1	0	1	Leave
E4	7	2	1	0	1	Leave
E5	4	6	0	0	0	Stay
E6	8	2	1	0	1	Leave

A new employee has:

Experience in Years	Satisfaction Score	Overtime	Promotion Received	Workplace Conflict
5	4	1	0	1

Using KNN with K = 3, classify the employee as Stay or Leave using all four distance measures.

ANSWER

(a) Features used

Numerical features: Experience in Years, Satisfaction Score

Binary features: Overtime, Promotion Received, Workplace Conflict

(b-e) Distance calculation (new sample = [5, 4, 1, 0, 1])

Sample calculation for E1 (the same formulas are applied to every training sample):

```
Euclidean(E1) = sqrt((5-2)^2 + (4-8)^2 + (1-0)^2 + (0-1)^2 + (1-0)^2) = sqrt(9 + 16 + 1 + 1 + 1) = sqrt(28) = 5.292
Manhattan(E1) = |5-2| + |4-8| + |1-0| + |0-1| + |1-0| = 3 + 4 + 1 + 1 + 1 = 10
Minkowski(E1,p=3) = (|5-2|^3 + |4-8|^3 + |1-0|^3 + |0-1|^3 + |1-0|^3)^(1/3) = (27 + 64 + 1 + 1 + 1)^(1/3) = 4.547
Hamming(E1) = compare binary new(1, 0, 1) vs E1(0, 1, 0) -> 3 mismatch(es) = 3
```

Distances from the new sample to all training samples:

Sample	Class	Euclidean (b)	Manhattan (c)	Minkowski p=3 (d)	Hamming (e)
E1	Stay	5.292	10	4.547	3
E2	Stay	4.000	8	3.362	3
E3	Leave	1.414	2	1.260	0
E4	Leave	2.828	4	2.520	0
E5	Stay	2.646	5	2.224	2
E6	Leave	3.606	5	3.271	0

(f) 3 Nearest Neighbours & (g) Majority-vote Prediction

Distance measure	3 Nearest Neighbours	Votes	Prediction
Euclidean	E3 (Leave), E5 (Stay), E4 (Leave)	Leave:2; Stay:1	Leave
Manhattan	E3 (Leave), E4 (Leave), E5 (Stay)	Leave:2; Stay:1	Leave
Minkowski p=3	E3 (Leave), E5 (Stay), E4 (Leave)	Leave:2; Stay:1	Leave
Hamming	E3 (Leave), E4 (Leave), E6 (Leave)	Leave:3	Leave

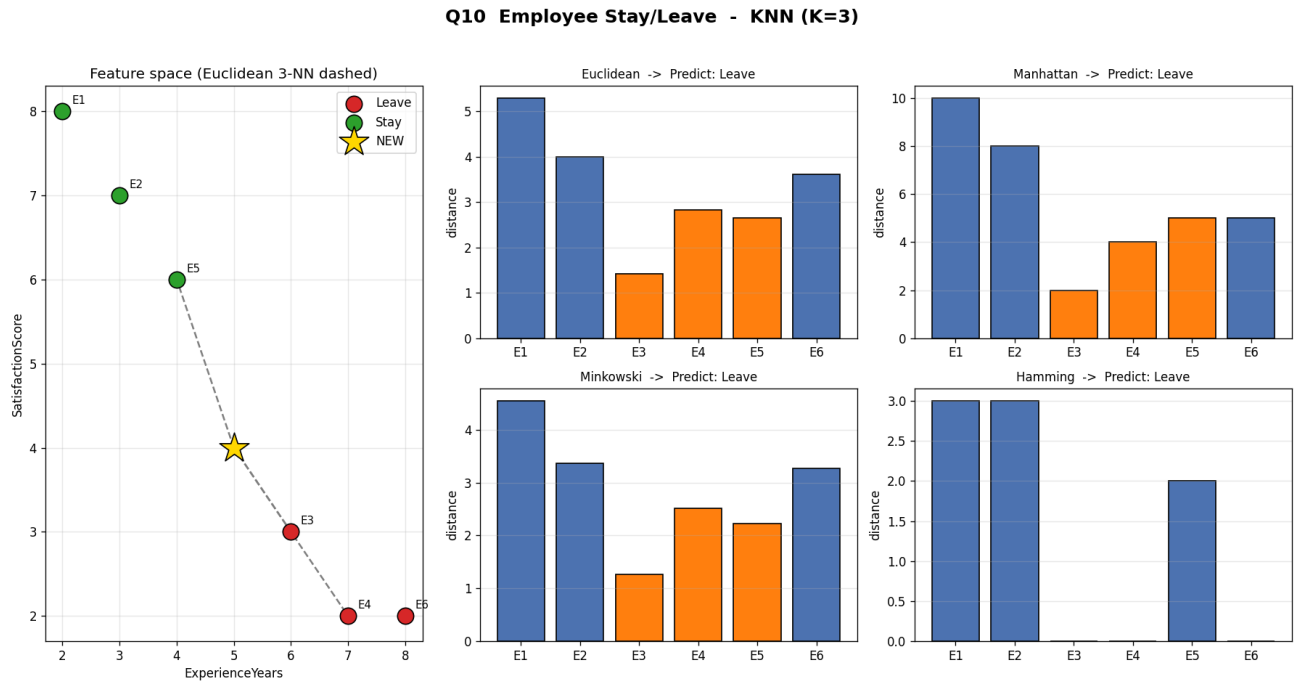
Python Code

```
"""
Question 10 : Employee Attrition Prediction (Stay / Leave)  --  KNN (K = 3)
------------------------------------------------------------------------
Input  : q10_employee.csv  (6 training rows + 1 NEW row)
Output : console results + a graph (q10_employee_knn.png)

Distance measures : Euclidean, Manhattan, Minkowski(p=3), Hamming
```

```
  * Euclidean / Manhattan / Minkowski  -> use ALL 5 features
  * Hamming                            -> use only the 3 BINARY features
"""


import numpy as np
import pandas as pd
import matplotlib.pyplot as plt


# ---------------------------------------------------------------- configuration
CSV_FILE   = "q10_employee.csv"
NUMERIC    = ["ExperienceYears", "SatisfactionScore"]
BINARY     = ["Overtime", "PromotionReceived", "WorkplaceConflict"]
LABEL_COL  = "Class"
K          = 3
P_MINKOWSKI = 3
TITLE      = "Q10  Employee Stay/Leave"
PNG_FILE   = "q10_employee_knn.png"


# ---------------------------------------------------------------- read csv input
df = pd.read_csv(CSV_FILE)
train = df[df["ID"] != "NEW"].reset_index(drop=True)
new   = df[df["ID"] == "NEW"].reset_index(drop=True)


ALL_FEATURES = NUMERIC + BINARY


print("(a) Numerical features :", NUMERIC)
print("    Binary   features  :", BINARY)
print("\nTraining data:\n", train[["ID"] + ALL_FEATURES + [LABEL_COL]].to_string(index=False))
print("\nNew sample:\n", new[["ID"] + ALL_FEATURES].to_string(index=False), "\n")


# ---------------------------------------------------------------- distances
X  = train[ALL_FEATURES].to_numpy(dtype=float)
q  = new[ALL_FEATURES].to_numpy(dtype=float)[0]
Xb = train[BINARY].to_numpy(dtype=int)
qb = new[BINARY].to_numpy(dtype=int)[0]


diff = X - q
euclidean = np.sqrt((diff ** 2).sum(axis=1))                      # (b)
manhattan = np.abs(diff).sum(axis=1)                              # (c)
minkowski = (np.abs(diff) ** P_MINKOWSKI).sum(axis=1) ** (1 / P_MINKOWSKI)  # (d)
hamming   = (Xb != qb).sum(axis=1)                                # (e)


results = pd.DataFrame({
    "ID": train["ID"],
    LABEL_COL: train[LABEL_COL],
    "Euclidean": euclidean.round(3),
    "Manhattan": manhattan.round(3),
    "Minkowski": minkowski.round(3),
    "Hamming": hamming,
})
print("(b-e) Distances to every training sample:\n", results.to_string(index=False), "\n")


# ---------------------------------------------------------------- 3-NN + voting
measures = {"Euclidean": euclidean, "Manhattan": manhattan,
            "Minkowski": minkowski, "Hamming": hamming}
predictions = {}


for name, dist in measures.items():
    order = np.argsort(dist, kind="stable")[:K]          # (f)
    neighbours = train.loc[order, "ID"].tolist()
    votes = train.loc[order, LABEL_COL].value_counts()
    pred = votes.idxmax()                                # (g)
    predictions[name] = pred
    print(f"{name:10s} -> 3 nearest: {neighbours}  votes: {dict(votes)}  => {pred}")


print("\n(g) Predictions:", predictions)


# ---------------------------------------------------------------- graphs
classes = sorted(train[LABEL_COL].unique())
palette = {classes[0]: "#d62728", classes[1]: "#2ca02c"}   # Leave red, Stay green


fig = plt.figure(figsize=(15, 8))
```

```
fig.suptitle(TITLE + "  -  KNN (K=3)", fontsize=15, fontweight="bold")
gs = fig.add_gridspec(2, 3)

ax = fig.add_subplot(gs[:, 0])
for cls in classes:
    sub = train[train[LABEL_COL] == cls]
    ax.scatter(sub[NUMERIC[0]], sub[NUMERIC[1]], s=180, c=palette[cls],
               label=cls, edgecolors="black", zorder=3)
for _, r in train.iterrows():
    ax.annotate(r["ID"], (r[NUMERIC[0]], r[NUMERIC[1]]),
                textcoords="offset points", xytext=(8, 6), fontsize=9)
ax.scatter(new[NUMERIC[0]], new[NUMERIC[1]], marker="*", s=650, c="gold",
           edgecolors="black", label="NEW", zorder=4)
for idx in np.argsort(euclidean)[:K]:
    ax.plot([new[NUMERIC[0]].iloc[0], train[NUMERIC[0]].iloc[idx]],
            [new[NUMERIC[1]].iloc[0], train[NUMERIC[1]].iloc[idx]],
            "--", c="gray", zorder=2)
ax.set_xlabel(NUMERIC[0]); ax.set_ylabel(NUMERIC[1])
ax.set_title("Feature space (Euclidean 3-NN dashed)")
ax.legend(); ax.grid(alpha=0.3)

positions = [gs[0, 1], gs[0, 2], gs[1, 1], gs[1, 2]]
for (name, dist), pos in zip(measures.items(), positions):
    axb = fig.add_subplot(pos)
    nearest = set(np.argsort(dist, kind="stable")[:K])
    colors = ["#ff7f0e" if i in nearest else "#4c72b0" for i in range(len(dist))]
    axb.bar(train["ID"], dist, color=colors, edgecolor="black")
    axb.set_title(f"{name}  ->  Predict: {predictions[name]}", fontsize=10)
    axb.set_ylabel("distance"); axb.grid(axis="y", alpha=0.3)

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.savefig(PNG_FILE, dpi=120)
print("\nGraph saved as", PNG_FILE)
plt.show()
```

Graph Output



(h) Comparison & Final Conclusion

- All four measures give LEAVE - unanimous.

- Low satisfaction (4) with overtime and workplace conflict matches the Leave cluster (E3, E4, E6); Hamming = 0 to every Leave record.

- Numeric dissatisfaction and the negative binary flags agree, giving a stable result across all metrics.

- Most suitable: the binary flags (overtime, no promotion, conflict) are the clearest attrition drivers (Hamming); Euclidean agrees.

Final Answer (majority vote): LEAVE